

Emotion Recognition In Speech

September, 2022

UNIVERSITY OF HERTFORDSHIRE

School of Physics, Astronomy and Mathematics

MSc Data Science

7PAM2002-0209-2021 - Data Science Project

SEPTEMBER 4

Authored by: Mahrukh Shabbir Hasan

Student ID: 20047638

Supervisor: Carolyn Devereux

MSc Final Project Declaration

This report is submitted in partial fulfillment of the requirement for the degree of Master of Science in Data Science at the University of Hertfordshire, Hatfield (UH).

Unless otherwise noted in the report, this is my original work.

In my MSc project, I didn't employ any human subjects.

I hereby authorise the report to be posted on the university website if the source is cited appropriately.

Acknowledgement

As I come close to finishing my post-graduate studies, I would like to emphasize that it has been a wonderful learning experience, and I want to express my gratitude to all the people who have supported me along the way.

I'd like to start by expressing my gratitude to Almighty God for never ceasing to inspire me with His endless blessings and for giving me the confidence and valor to move forward with assurance and self-belief.

I would like to convey my appreciation and gratitude to Carolyn Devereux, who served as my supervisor, for her constant advice and assistance in this project. I am appreciative of her constant support and her patience towards my inquisitiveness.

I would also like to express my gratitude towards all my professors at the University of Hertfordshire who helped me gain knowledge and understanding of the subjects and helped me throughout my course.

I would also like to thank my parents, my sister, and my friends for their unwavering encouragement and support, without which this would not have been possible.

Abstract

The main objective of this project was to conduct comparative research on the topic of SER. The models explored in this project were Convolutional Neural Networks, Multi-layer Perceptron, Support Vector Machines and Long-Short Term Memory (LSTM). These models were taken to establish comparison of performance metrics of classical classifiers versus machine learning classifiers. The data set utilised in the project was a combination of CREMA-D (Crowd-Sourced Emotional Multimodal Actors Dataset) and TESS (Toronto Emotional Speech Set) along with Mel-frequency Cepstrum coefficients (MFCC) which served as method of extraction of features or information. This project concluded that the LSTM was the most suitable with respect to accuracy with 91% training accuracy and 98% test accuracy whereas MLP gave the most reliable accuracy vs testing time balance with less than a second of testing time. This project highlights the model usability in real-time applications provided the drawbacks of the data available in real-time and also the limitations of models.

Contents

1. Introduction	8
2. Literature Review	12
3. Methodology	17
3.1 Brief Overview:.....	17
3.2 Dataset Used:	17
3.3 Data Pre-Processing:	18
3.3.1 Feature Extraction:	20
3.3.2 Data Augmentation:	22
3.4 Classification Models:.....	23
3.4.1 Convolutional Neural Network:	23
3.4.2 Multi-Layer Perceptron:	25
3.4.3 Long Short-Term Memory (LSTM):	26
3.4.4 Support Vector Machines:	29
4. Results	31
5. Discussion of Results	39
5.1 Comparison of models:	39
5.2 Comparison with Other Papers:	39
5.3 Applying Models to some of the Applications:	41
5.4 Improvements of Models:	41
5.5 Limitations:	42
6. Conclusion.....	43
7. References	45
8. Appendix.....	50

List of Figures:

Figure 1: Depiction of Speech Emotion Recognition System.....	9
Figure 2: Growth of SER in market value.....	12
Figure 3: Overview of SER.....	13
Figure 4: Flowchart of the SER system in this project.....	17
Figure 5: Waveforms of two Audio Signals.....	18
Figure 6: Flowchart of SER systems in general.....	19
Figure 7: Visual of Audio Signal.....	20
Figure 8: Flowchart of MFCCs.....	21
Figure 9: Visual representation of CNN model.....	24
Figure 10: ReLU function.....	25
Figure 11: Representation of MLP model.....	26
Figure 12: RNN Model.....	27
Figure 13: LSTM Model with Input, Output and Forget gates.....	28
Figure 14: Visual SVM Kernels Example.....	30
Figure 15: Plot of Accuracy and Loss of CNN model on training and validation data.....	31
Figure 16: Plot of Accuracy and Loss of RNN model on training and validation data.....	33
Figure 17: Precision and recall.....	34
Figure 18: Classification report and heatmap of SVM.....	35
Figure 19: Classification report and heatmap of MLP.....	35
Figure 20: Classification report and heatmap of RNN.....	35
Figure 21: Classification report and heatmap of CNN.....	35
Figure 22: Statistics of Each Emotion.....	38

List of Tables:

Table 1: Research papers and their comparison.....	15
Table 2: Architecture of CNN model.....	25
Table 3: Architecture of RNN model.....	28
Table 4: Results of each model.....	31
Table 5: Different versions of MLP models built in this project.....	32

1. Introduction

Humans express themselves most naturally through speech. We are so reliant on it that we understand its significance even when using other forms of communication, like emails or messages. Hence the use of emojis in our text messages has become predominant. Emojis have proliferated in text messaging, which is not surprising given the potential for misunderstanding, making it essential to convey the underlying emotions of the context through text. According to Kerkeni et al., (2020), emotion is a key factor in our everyday interpersonal relationships. It is essential for our rational, logical and wise/intelligent decisions. We can align and comprehend others' emotions by communicating our own emotions with them and communicating feedback or reciprocating their sentiments. Since emotions make it easier for people to comprehend one another, it seems natural to apply this idea to computers as well. In the modern world, if the same level of understanding as human-human interaction could be achieved by human-machine interaction, it would represent an enormous advancement in technology.

Finding the speakers' emotional state depends on a variety of factors. According to Cowie et al., (2001), if the emotional state of the individual can be precisely determined during human-machine interaction, the computer applications can be made to deliver better appropriate responses. Most contemporary automatic speech recognition systems tend to use natural language comprehension and processing known as NLP (Natural Language Processing) to further improve voice recognition accuracy. The accuracy of the system can be increased by extracting the speaker's emotional state, which will also help with language interpretation with far better accuracy. Translation is generally necessary to conduct multilingual communications. The semantic component of speech is the primary focus of current machine translation algorithms, but emotions can be recognized or rather, learnt from the features extracted from the speech/audio to achieve the desirable level of accuracy of the translation systems (Cowie et al., 2001).

Therefore, the idea of SER (Speech Emotion recognition) came into existence. SER is basically an attempt to recognize human emotions from speech/audio. The fundamental principle is the fact that voice frequently conveys the underlying emotion through pitch and tone. According to (Femia, 2020), this same method along with the facial expression is used by animals like horses to be able to understand human emotion. It is crucial to have an emotional detection framework, which consists of different modules that carry out tasks like audio to text conversion, followed by feature extraction from the audio files, then the selection of features from those extracted features, and finally

classification of those features to determine the underlying emotions in those audio files (Sudhakar and Anil, 2015). Figure 1 below depicts a typical SER system.

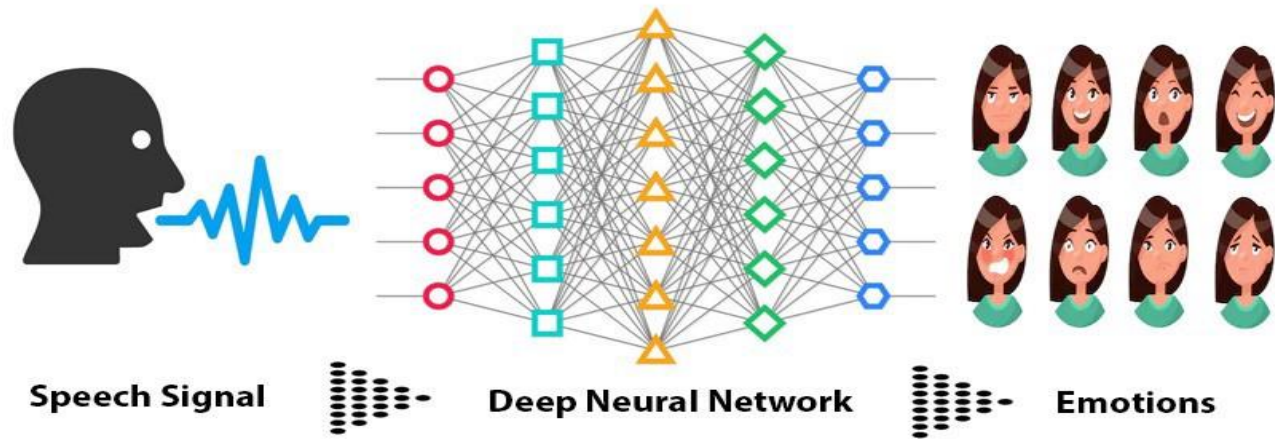


Fig 1. Depiction of Speech Emotion Recognition System (Source: MevonAI, <https://tinyurl.com/ze94b7ud>)

Detection of human emotions through a machine has many applications. For example, some of the applications according to Kerkeni et al., (2020) are, detection of emotion in the audio surveillance, business applications, clinical trials, interfacing with robots, web-based E-learning, entertainment sector, emotion detection in call centers, banking sector in detection of fraud, video games, etc. Thanks to smartphones that can synthesize or rather, transcribe voice commands, speech recognition has already been part of our daily lives. As an advancement to this, smartphone companies could also employ the Speech Emotion Recognition (SER) model to be able to recognize our emotions in real-time. (Akçay and Oğuz, 2020).

These SER methods could also be used to watch how businesses engage with their clients in call centers. Currently, a human professional with limited skills must be included in order to assess emotions in these talks. However, using machines to complete the process of assessment will prove to be far more cost-effective and produce results that are a lot more reliable. Such a strategy might also be advantageous for public services. For instance, it would be feasible to examine the emotions expressed in the voices, or speeches, of members of parliament or politicians. As the attitudes and sincerity of politicians may be looked into and further analyzed, such knowledge could be of great interest and benefit to society. Additionally, other NGOs tackling problems in civil society could apply emotion recognition. For instance, it will be possible to follow the emotional states and behavior of various social groups by utilizing emotion recognition in speech (Lugovic, Dunder and Horvat, 2016). Additionally, these methods could be applied to academic research, particularly social science research, to reach a higher level of granularity. For instance, when conducting interviews, one could pick up on emotions in

speech. This method would allow further scope for the analysis of another speech utilizing the same data collection and processing technique.

On a more societal level, emotion recognition could be used to enhance the day-to-day interaction between artificial intelligence and humans. Siri and Alexa, two examples of artificially intelligent devices, are at present programmed to play music or comprehend spoken commands from humans. Speech Emotion Recognition can evaluate a person's mood and, when used in conjunction with IOT (Internet of Things), change the music and lighting of the place to elevate mood. If a sad/depressing emotion is identified, it can also recommend some shows to assist people feel better. In the medical field too, the emotional state of the patient can be tracked and monitored and based on that, suitable medical treatment can be administered. Another application would be to teach computers to recognize the emotional expressions made by humans, so that tailored email answers could be sent out in response.

Over the past few years, this subject has drawn more and more attention from researchers and recent advances in the field of artificial intelligence and deep learning techniques have further intensified the interest in this topic. With many researchers coming up with different algorithms and ML (Machine Learning) models to try and achieve acceptable accuracy, there is still a lot of scope for research in this area. Although many models have been created for SER, the accuracy and the predictive quality of the models have not quite reached our expectations to be able to be deployed in real-time applications.

The objective of this project is conducting comparative research on the topic of detecting emotion in speeches using the four commonly used models – Convolutional Neural Networks (CNN), Support Vector Machines (SVM), Multi-Layer Perceptron (MLP), Recurrent Neural Networks (RNN)- Long Short-Term Memory (LSTM) and comparing their accuracy, loss functions, training/testing speed and other parameters when trained on the same dataset. The goal of this project is to understand various factors affecting the overall model performances of SER system and generalization of SER systems to evaluate the feasibility of deployment of the SER system in real-time applications. Although researchers have used varied datasets for their models along with various methods of pre-processing, the datasets CREMA-D and TESS were opted for this project as these datasets have large number of audio files with a total of 7 emotions with varied voices of actors both male and female of different range of age category. These datasets will be discussed in detail in the Section 3.2. Although different research papers studied

and built different models in their research, the models – CNN, MLP, SVM and RNN-LSTM were specifically chosen for this project because:

CNN: was used as it uses convolutional layers with the combination of pooling layers which act as filters to reduce low-pixels or in this case, noise from the data. It also helps in dimensionality reduction.

LSTM: was used due to its ability to predict sequential data.

MLP: is mainly used for classification problems, and SER being one, MLP was chosen.

SVM: is considered to be one of the best classifiers with optimal margin. This helps classification of a new datapoint a lot easier. SVM also has the benefit of not requiring extensive preprocessing. Hence SVM was chosen.

Further information of each model is detailed in Section 3.4 of this report.

"Machine learning allows us to build software solutions that exceed human understanding and shows us how AI can innervate every industry." - *SpaceX and Tesla board member, Steve Jurvetson*

2. Literature Review

SER is a widely researched topic which has garnered a piqued interest over the former years with an increasing demand for not just research but the market as well. According to the survey by Anderson, (2022) via the site marketandmarkets.com, an increasing exponential trend for the demand of SER was observed. With that observation, an estimate of \$56 billion is predicted in 2024 from \$21.6 billion in 2019 which is more than double in just 5 years implying the extend of its captivity amongst people. Figure 2 depicts the predicted growth of SER over the years for different regions.

EMOTION DETECTION AND RECOGNITION MARKET, BY REGION (USD BILLION)

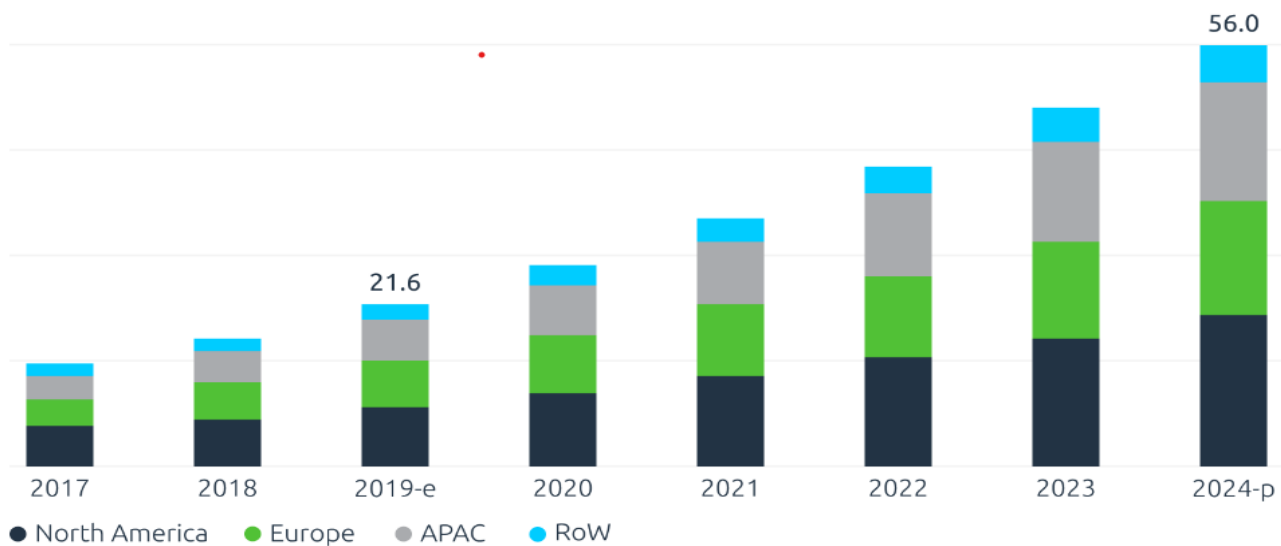


Fig. 2: Growth of SER in market value. (Source: <https://www.iflexion.com/blog/emotion-recognition-software>)

Some of the major tech companies in the market have already started deploying SER systems. According to (RESEARCH, 2022), the companies Amazon's AWS, Alphabet Inc's Google services, Apple, Microsoft, IBM have already started deploying SER systems in small scale. One of the top companies in the voice recognition market is LumenVox LLC, a firm with its headquarters in San Diego, California, USA. Another company, sensory Inc, uses neural network, NLP and ML technologies to enrich the user interfaces. The company offers a machine learning technique for speech and emotion recognition primarily in software and electronic devices.

From a research standpoint, researchers have used different models on different datasets to achieve different accuracies and model performances. A very broader picture of overview of SER provided by (Akçay and Oğuz, 2020) is shown in Figure 3:

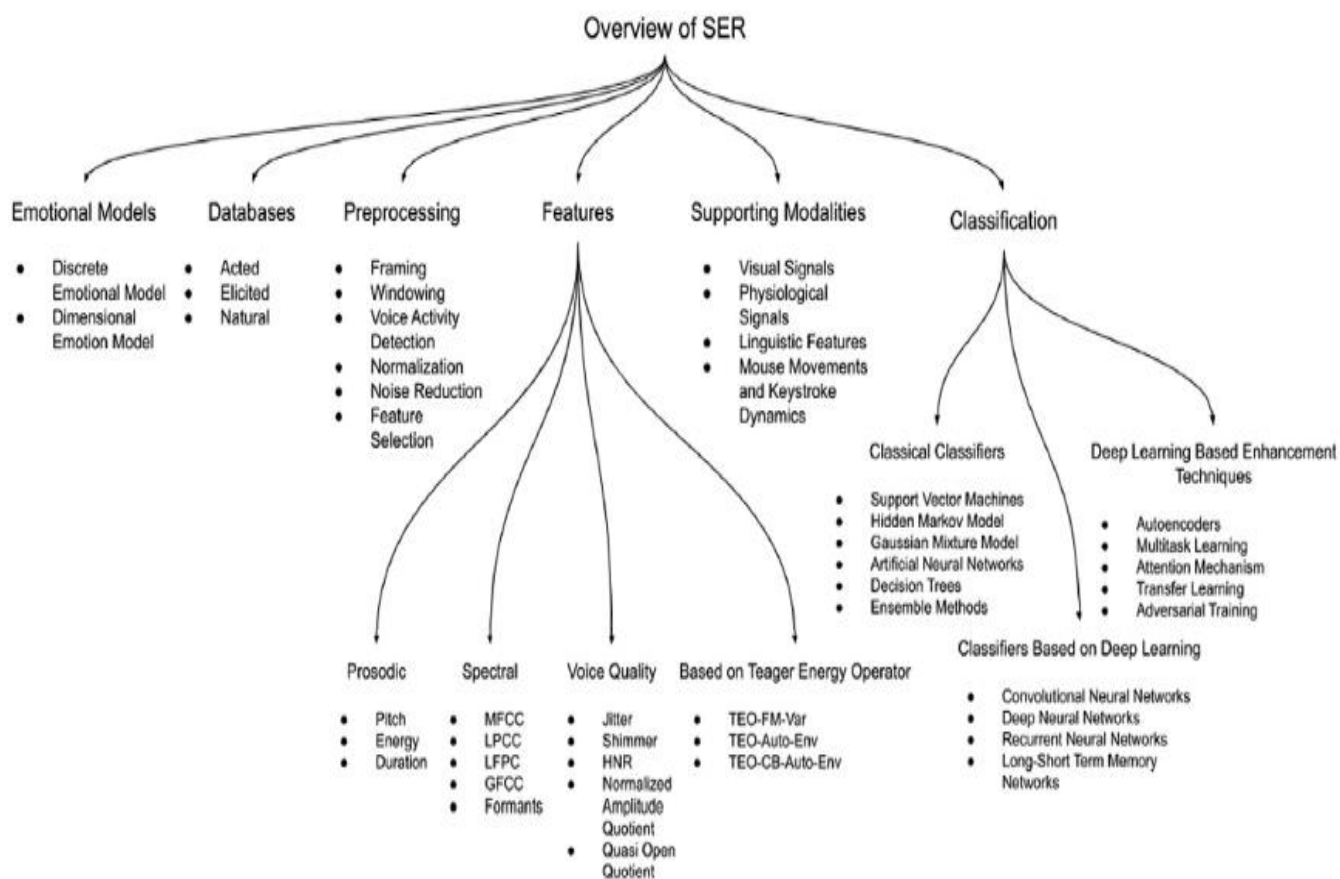


Fig 3: Overview of SER. (Source: Akçay and Oğuz, (2020))

The overview of SER shows the current research done and the information available around SER. The research by Akçay and Oğuz, (2020) mainly focuses on the databases, the preprocessing techniques and the classifiers that have been used for SER. Databases like RAVDESS, SAVEE, CREMA-D, TESS, IEMOCAP, MES, DES etc. are widely used in research. These datasets contain the recording of different emotions acted out by professionals in different languages. There are many other datasets available for generic speech recognition as well. Preprocessing techniques are used for extracting features from the given voice recordings. The list of preprocessing techniques as listed in the Figure. 3, range from framing, windowing to normalization. The techniques depend on the type of features that are to be extracted. At first, researchers commonly used prosodic features like pitch, energy and duration of the voice recordings like in Nwe, Foo, and De Silva, (2003) and Mao et al., (2014). Spectral features have now seemingly become widely accepted (example papers: Issa, Fatih Demirci and Yazici, (2020), Fayek, Lech and Cavedon, (2017), Kerkeni et al., (2018)) as a result of their capacity to increase the performance of an algorithm. This report uses spectral features for model training and hence MFCCs are extracted using windowing. As for the classification techniques,

the list broadly categorizes the techniques into classical classifiers, deep learning classifiers and enhanced deep learning classifiers. At the beginning, during the late 90s, the classical classifiers were popular amongst the research (example paper: Nwe, Foo, and De Silva, (2003), which used Hidden Markov Model) but after the materialization of deep learning specially after the release of Tensorflow and Keras libraries in 2015, many researches researching around this topic in early 2010s shifted to CNN, RNN-LSTM and DNN (example papers: Mirsamadi et al., (2017), Sai, et al., (2022), and Fayek, Lech and Cavedon, (2017)). There is still a huge scope for research in the enhanced deep learning techniques which may become popular techniques for SER in the coming years.

There are many other papers that assess the current research on speech emotion recognition. Many researchers have proposed multiple solutions using various mathematical algorithms and deep learning models. Some of them worth mentioning and comparing are shown in Table 1.

Papers	Classifiers	Dataset Used	Results
Speech Emotion Recognition: Methods and Cases Study. (Kerkeni et al., 2018)	Multivariate Linear regression, SVM, RNN	Berlin and Spanish Emotional database	Highest results for each classifier: MLR – Spanish – 82.41%, Berlin – 75% SVM – Spanish – 77.63%, Berlin – 63.30% RNN- Spanish – 90.05%, Berlin – 69.55%
Speech emotion recognition using hidden Markov models. (Nwe, Foo, and De Silva, 2003)	LFPC to represent speech signals and Hidden Markov Models were used as classifiers	A user defined dataset, with voices of Burmese and mandarin speaking people. A total of 720 voice recordings.	An average accuracy of 77.1%, 89% when emotions were identified individually
Evaluating deep learning architectures for Speech Emotion Recognition. (Fayek, Lech and Cavedon, 2017)	DNN – CNN + LSTM	IEMOCAP	The best accuracy achieved was 64.78%. Preprocessing was done with hamming window and log FFT.
Speech emotion recognition: Emotional models, databases, features, preprocessing methods, supporting modalities, and classifiers. (Akçay and Oğuz, 2020)	HMM, SVM, CNN and many more	No Dataset used	Just comparative research on what has been done by other authors
Learning salient features for speech emotion recognition using convolutional neural networks. (Mao et al., 2014)	CNN	SAVEE, Emo-DB (Berlin), DES (German), MES(Mandarin)	73.6% accuracy for SAVEE DS, 85.2% for EMODB, 79.9% for DES DS 78.3% for MES DS

Automatic speech emotion recognition using recurrent neural networks with local attention. (Mirsamadi et al., 2017)	RNN	IEMOCAP	61.8% recognition rate with raw features 63.5% recognition rate with LLD features.
Emotion recognition and its application to computer agents with spontaneous interactive capabilities. (Nakatsu et al., 1999)	Neural Networks	User-defined dataset	50% accuracy of recognition
Adieu features? end-to-end speech emotion recognition using a deep convolutional recurrent network. (Trigeorgis et al., 2016)	Deep CNN with LSTM	RECOLA	Highest valence – 0.261
Speech emotion recognition with deep convolutional neural networks (Issa, Fatih Demirci and Yazici, 2020)	CNN	RAVDESS, EMODB, IEMOCAP	RAVDESS - 71.61% EMODB – 86.1% IEMOCAP – 64.30%
Speech Emotion Recognition using LSTM and RNN (Sai, et al., 2022)	RNN LSTM	The dataset used was RAVDESS	accuracy was 95% overall

Table 1: Research papers and their comparison

From the papers in Table 1, it would be relevant to discuss the papers and their achieved accuracy in order to incorporate their work into this project. According to this project, the accuracy of the models heavily depends on the dataset which is being used and the type of preprocessing done on that dataset to extract information. As shown in Table 1, the relatively older papers used mathematical algorithms to develop SER like in paper Speech emotion recognition using hidden Markov models (Nwe, Foo, and De Silva, 2003) whereas the latest papers focused on neural networks for it like papers Trigeorgis et al., (2016), Sai, et al., (2022) and Issa, Fatih Demirci and Yazici, (2020). While CNN model was used by many of the researchers, RNN proved to be delivering the expected results (Sai, et al., 2022). SVM model, being widely used for classification algorithms (Kerkeni et al., 2018), also provided satisfactory results for many of the datasets used. From the preprocessing standpoint, one of the crucial processes in the creation of an SER system is feature extraction. Due to its importance, many researchers have used different techniques and methods to extract features with higher accuracy and better overall performance. Some of the common techniques used according to (Akçay and Oğuz, 2020) were Modulation spectral (MS) features (E.g., Kerkeni et al., (2018)), Mel-Frequency Cepstrum Coefficients (MFCC) (E.g., Issa, Fatih Demirci and Yazici, (2020)),

Kerkeni et al., (2018)), Low-Level Descriptors (LLD) features (E.g., Mirsamadi et al., (2017)), Linear Prediction Coefficients (LPC) features (E.g., Shen, Changjun and Chen, (2011)) etc. These techniques helped with extracting most common features like pitch, spectral features and energy. As observed from this project, many researchers favored the MFCC technique of extracting the features as it yielded far better results when compared to other techniques.

Not only was there abundant published literature found for this topic, but also this topic has taken the masses by sense of intrigue. Many competitions were held for developing the SER with many people coming up with different algorithms and deep learning models to achieve a reliable accuracy. Again, the most common technique of feature extraction in these codes was found to be MFCC. While many leaned towards CNN model, MLP was also one of their favorite models. Then, there were few who tried replicating the results of the published papers. The website by Meta AI, <https://paperswithcode.com/task/speech-emotion-recognition>, gives the list of papers generally referred to and the relevant code that people tried for those papers.

In this report, the overall performance of each model (CNN, MLP, SVM, RNN-LSTM) when trained and tested against the same data with same feature extraction technique is highlighted. In the Section. 4 of this report, more specifics about this study are covered.

3. Methodology

3.1 Brief Overview:

This project was built in Google Colaboratory in Python language using various libraries for models used. The libraries numpy, pandas, matplotlib were used for basic mathematical, visualization and Dataframe operations. The libraries tensorflow, keras and sklearn were used for model building. Librosa library was used for preprocessing data using the MFCC technique. The first step was to extract the datasets CREMA-D and TESS from the Kaggle website. The data was downloaded into the google colab environment and a Dataframe containing the information from these two datasets was constructed. The Dataframe was made up of the audio path and the associated emotions. Using the data in this Dataframe the features were extracted and stored into another Dataframe, the specifics of this process will be covered in the Section 3.3 of this report. The next step was data preparation which included scaling of data features, one-hot encoding the emotions which were in form of words, and splitting data into training and test datasets. to make them more compatible with the models. Next came the building of the models CNN, MLP, SVM and RNN-LSTM. The data was then fed into these models for training the models. Parameters were calibrated to achieve desirable levels of accuracy and speed and the models were then tested against the test dataset. Figure 4 shows the flowchart of the tasks carried out in this project.

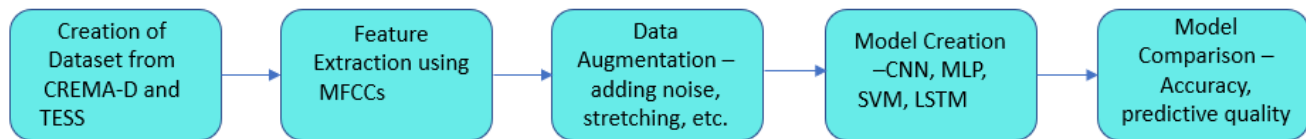


Fig 4: Flowchart of the SER system in this project

3.2 Dataset Used:

For this research topic, I integrated the data from two well-known datasets commonly used in SER: TESS (Toronto Emotional Speech Set) and CREMA-D (Crowd-Sourced Emotional Multimodal Actors Dataset) into a single Dataframe, which had a total of 10242 audio recordings and a total of seven emotions.

CREMA-D: Crowd-sourced Emotional Multimodal Actors Dataset

The CREMA-D dataset contains a distinct range of emotional states – sad, happy, neutral, anger, fear and disgust. It has about 7442 audio recorded files of 91 actors in total, of both male and female genders with ages between 20 to 74, with diverse ethnic

backgrounds. The dataset has voice recordings of 48 males and 43 females. The actors in the dataset have spoken 12 different sentences with different emotions.

TESS - Toronto Emotional Speech Set

The TESS dataset has about 2800 audio files of 2 women actors of age 26 years and 64 years enacting emotions – fear, happy, sad, disgust, anger, surprise and neutral. This dataset has an extremely high quality of audio which gives me a superior quality dataset for my analysis.

To understand how the audio data is represented in amplitude-time domain, waveforms of two audio data along with the labeled emotions are as shown in Figure 5 below:

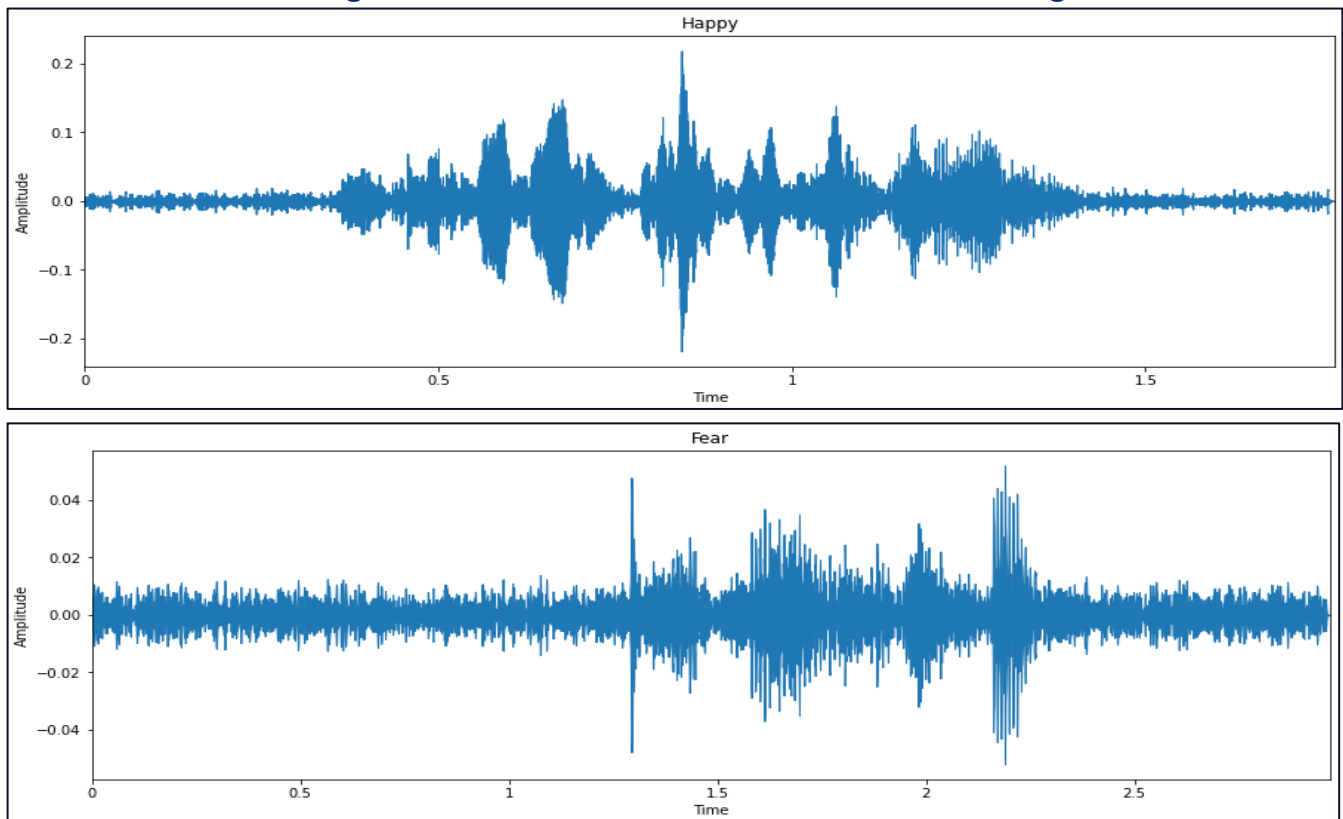


Fig 5: Waveforms of two Audio Signals

3.3 Data Pre-Processing:

The real-time/real-world data in the form of audios, images, videos, texts etc., is not the ideal data on which the machine learning models can be trained. Not only may it contain inconsistencies and at times, errors, but also it is often incomplete, and does not have a regular or a uniform structure. Preprocessed data can occasionally outweigh the most potent/powerful deep learning algorithms, to the point where machine learning models trained on subpar or inaccurate data could actively impair the study of researchers by

producing incorrect, inaccurate and unreliable conclusions/results of the research. (Mesevage, 2021). A general flow of SER systems is detailed in Figure 6.

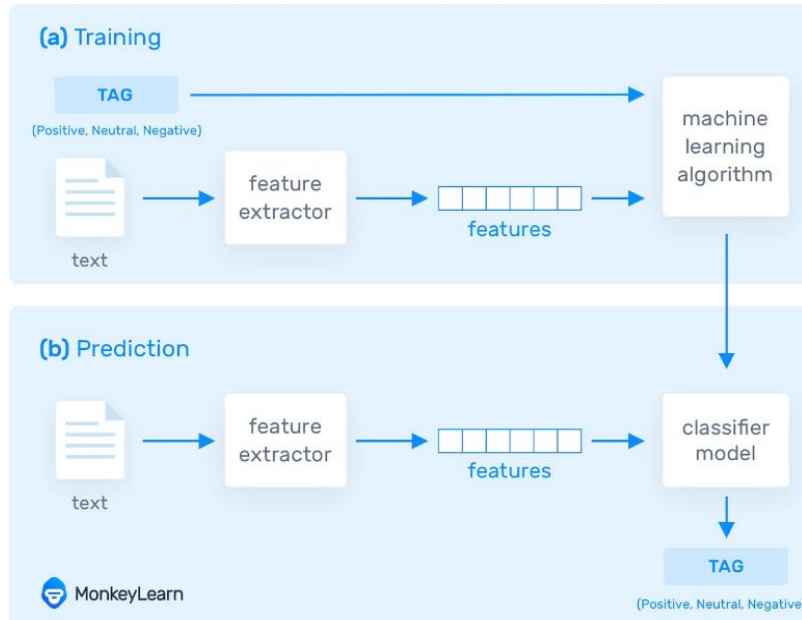


Fig 6: Flowchart of SER systems in general. (Source: <https://monkeylearn.com/blog/data-preprocessing/>)

For preprocessing my dataset, it was necessary to understand the nature of data and the various available procedures to extract information from it. Looking at the physiological perspective during the generation of speech, Williams and Stevens, (1981) specified that emotions such as anger, fear, or joy cause the sympathetic nervous system to become dominantly active. Increased blood pressure, heart rate, occasional dry mouth, and intermittent muscle tremors are the results one goes through while feeling those emotions. In line with this, speech is also spoken quickly, enunciated at times, and with powerful high frequency energy. On the other hand, when the parasympathetic nervous system is aroused, as when one feels sad, the blood pressure and heartbeat rate drop while the rate of generation of saliva rises, resulting in speech that is sluggish and lacking in high frequency energy. The total energy, energy distribution across the frequency spectrum, the frequency and length of speech signal and the pause duration of the speech, all exhibit the corresponding impacts of these physiological changes on speech (Nwe, Foo and De Silva, 2003). But this is not just limited to human psychology and this same concept of understanding the energy distribution in frequency spectrum can be extended to machines as well. Hence, taking this into consideration, preprocessing of data (which is to extract features by extracting information of audio signals in frequency and time domain) was done in this project. Further details of the feature extraction process in discussed in Section 3.3.1 and 3.3.2 below.

3.3.1 Feature Extraction:

Since our data is in audio format, preprocessing is the very first step after collection of data that is used to train the classifier models. In order to analyze and discover relationships between many entities, mainly the audio files, feature extraction from these audio signals becomes a crucial step which is used by many researchers in their feature extraction process. As shown in Figure 7, the audio signal can be disintegrated into a three-dimensional signal. The axes of which represent amplitude, time and frequency.

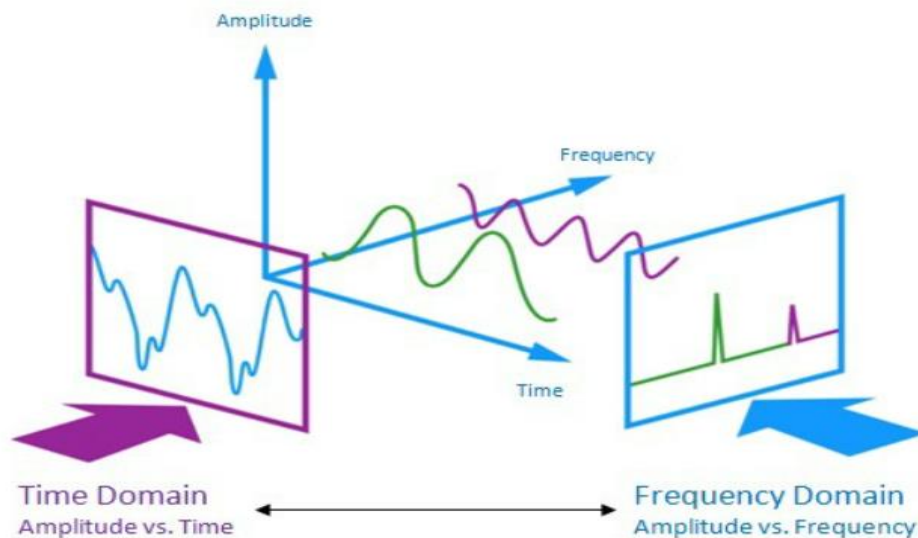


Fig 7: Visual of audio signal (Source: <https://www.kaggle.com/code/shivamburnwal/speech-emotion-recognition/notebook>)

To extract the features/information from these signals, MFCC or Mel-frequency Cepstrum coefficients feature extraction technique is used. The MFCC feature helps to extract significant audio information more effectively by representing the short-term power spectrum of the speech signal. Extraction of the Mel-frequency (where Mel is the scale for measuring pitch by listeners who are equidistant from one another) coefficients from these signals constitute the feature extraction process.

Mel-frequency Cepstrum coefficients:

MFCCs have been employed in the majority of recent studies to extract emotional features that are then fed into a classification algorithm. The most prevalent way to characterize a voice signal's spectral characteristics/property is using the Mel-frequency Cepstrum Coefficient. According to Kerkeni et al., (2018), these are the finest and highly reliable for voice recognition because they take into account how sensitively humans perceive frequencies. There are different versions of the MFCC transformations, but the typical block diagram of a basic MFCC is as below in Figure 8:

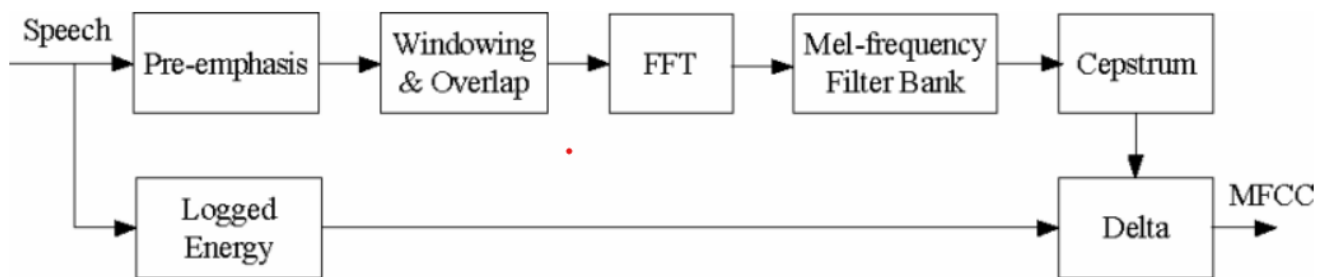


Fig 8: Flowchart of MFCCs. Source: <https://ieeexplore.ieee.org/abstract/document/1692543>

Pre-emphasis amplifies the energy in the higher frequency by a significant amount. When we analyze the audio signal's frequency domain, the energy at a higher frequency is much smaller than the energy at a lower frequency for voiced segments, such as vowels. The main objective of performing pre-emphasis is to improve the model's performance and accuracy by increasing the energy at higher frequencies. (Kiran, 2021).

Windowing breaks the audio signal into different segments to extract the required features. If the signal is chopped off at its edges directly during the signal splitting, it would result in the abrupt drop in amplitude of the signals at the margins leading to the generation of noise signals in the range of high frequency. Hence, instead of employing a rectangular window for windowing, a Hanning window (a sinusoidal wave which touches zero at both ends after removal of discontinuity) or a Hamming window (a sinusoidal wave which does not touch zero at both ends and has slight discontinuity) is used which helps in prevention of noise in the high-frequency range (Kiran, 2021).

FFT or the Fast Fourier Transform is used for the conversion of the incoming signal from the time domain to the frequency domain. This step is crucial because for audio signals, it is significantly simpler to analyze the data in the frequency domain than in the time domain (Kiran, 2021).

MEL-frequency filter bank uses the Mel scale to convert the actual frequency to a frequency that people can generally perceive. This is done because there is a difference in the way our ears perceive the sound versus the machines perceiving the sound. Our hearing is dependent on the fact that at lower frequencies than at higher frequencies, human beings have better sound or acoustic resolution. (Kiran, 2021).

Cepstrum was actually used for studying echoes, but its purpose is to detect the pitch in audio files. Cepstrum is basically a spectrum of a spectrum. In mathematical terms, it is

the inverse Fourier transform of the power spectrum which is obtained from the Discrete Fourier Transform (DFT) of the speech signal **(Kiran, 2021)**.

Performing these steps gave me the required features in the form of Mel frequency coefficients. But before data features are extracted, another procedure of data pre-processing is most commonly applied which is the technique of augmenting data known as data augmentation.

3.3.2 Data Augmentation:

A collection of techniques known as "data augmentation" are used to generate extra data points from existing data in order to artificially increase the volume of data. This includes making minor adjustments to the data or creating new data points using deep learning models. Applications for machine learning are always expanding and diversifying, particularly in the deep learning field. The obstacles that the field of artificial intelligence faces may be overcome with the help of data augmentation techniques. By creating additional and distinct instances for training datasets, data augmentation helps machine learning models perform better and produce better results. If the dataset is substantial and well-curated (clean) and has real-time accurate data which is taken from or closely resembling to the real-world data, a machine learning model will function better and be more accurate. For machine learning models, labelling and collecting data can be time-consuming and expensive for companies. By changing datasets using data augmentation techniques, businesses can reduce these operational costs. Data cleaning is one of the processes in creating a data model, which is important for high accuracy models. The model cannot make accurate predictions for inputs from the actual world, though, if cleaning decreases the representability of the data. Data augmentation approaches/techniques produce variances of data that the machine learning model could come across in the real world, making machine learning models more resilient and robust in nature **(Takimoglu, 2021)**.

As discussed previously, because the classifier models could not directly interpret or process the data from our audio recordings, feature extraction was utilized to transform the data into a format that could be handled by the classifier models. Therefore, in this project, the features were extracted using MFCCs after performing the data augmentation steps. The sequence of which is:

1. Addition of noise which is random addition of data points to the existing data.
2. Stretching of the audio signal which is to increase rate of flow of data.
3. Shifting of the audio signal in amplitude domain similar to normalization of data signal.

-
4. Pitch shifting of the audio data by a pitch factor of 0.7 (or simply the steps of 0.7).
 5. Speeding up the data in time domain by the speed factor of 1.25 (increasing the speed of the audio signal to 1.25 times).
 6. Speeding down the data in time domain by the speed factor of 0.75 (decreasing the speed of the audio signal to 0.75 times).

3.4 Classification Models:

For this project, the algorithms chosen were CNN, RNN-LSTM, MLP and SVM. These models were built and then trained and tested with the aforementioned predefined dataset containing extracted features from CREMA-D and TESS audio files. These models were taken to establish comparison of performance metrics of classical classifiers (SVM and MLP) versus deep learning classifiers (CNN and LSTM). But more specifically, these models were taken due to their unique characteristics. RNN was considered as it works well with sequential data and since the speech audio signal involves time domain information, RNN would be the right fit. MLP was considered due to its ability to perform classification tasks. SVM was considered due to its advantage of having optimal margin division for classification. SVM also has an advantage of less preprocessing involvement hence reducing the time involved in extensive preprocessing. CNN was considered due to its ability to filter out excessive noise and unwanted signals. CNN helps in retention of only important information by performing dimensionality reduction. These models are further detailed in sections 3.4.1, 3.4.2, 3.4.3, 3.4.4 below.

3.4.1 Convolutional Neural Network:

The most advanced and inventive models used for taking raw, low pixel data as input and then extracting high-level features as output are CNNs. These features are extracted from images using CNN's large number of kernels, and then the CNN model is trained using these extracted features to handle important classification tasks. The architecture of CNN is an amalgamation of three components; the first layer is called convolutional layer, which may filter input using a number of filters. A single convolutional layer is created after each filter evaluates the input by using the dot product and submission procedure to build several feature maps. Next comes the pooling layer which constitutes as the second component in CNN architecture. This is used to downscale or in other terms, minimize the number of dimensions in feature maps. Based on the requirements, various methods can be employed to reduce dimensionality of the data like; mean pooling, min pooling, average pooling, max pooling etc. The last component, CNN's fully connected layers (FC), is mainly used to extract the global characteristics, which are then provided to a SoftMax classifier to calculate the likelihood for each class/group i.e. its

probability. All of these layers in the given sequence—Convolutional Layers (CL), Pooling Layers (PL), Fully Connected Layer (also known as the Dense Layer), and SoftMax Classifier—are arranged by a CNN in a hierarchical structure (Mustaqeem and Kwon, 2019). The SoftMax function can be used to change a vector of K real values into a vector of K real values that equal 1. The SoftMax converts input values, which may be zero, negative, positive, or greater than one, into numbers in the range of 0 and 1, making them understandable as probabilities. The result of the SoftMax function provides you the likelihood that the output returned belongs to any of the classes, which is comparable to a categorical probability distribution. Apart from these, a CNN layer must also contain activation function for the layers its built of. An activation function in an artificial neural network terminology, is a task that yields a reduced output for minuscule inputs and a larger output if the inputs are greater than a tolerance level. If the inputs are sufficiently large, the activation function "triggers"; otherwise, nothing happens. To put it another way, an activation function functions like a barrier that verifies that if an incoming value is greater than a threshold value.

CNNs are ideal for image recognition and classification but have proved to be classifiers for emotion in speech as well. A basic example of CNN architecture is shown in Figure 9.

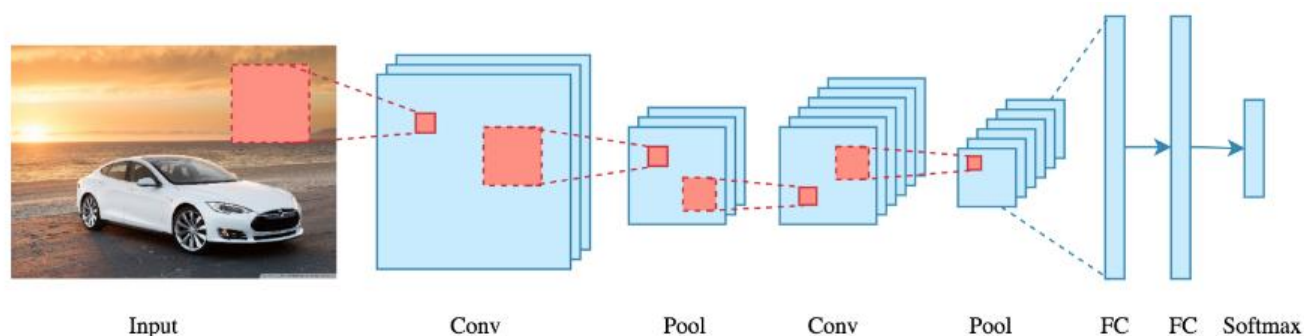


Fig 9: Visual representation of CNN model. (Source: <https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2>)

In this project, a CNN model was built with the convolutional, max pool and dense layers. The CNN model was built by setting up the combination of convolutional layers with the activation being ReLU (Rectified Linear Unit) along with max-pooling layers. The plot of a ReLU graph is as shown in Figure 10. ReLU works on the formula:

$$\begin{aligned} \text{If input} \leq 0, \text{ReLU(input)} &= 0 \\ \text{If input} > 0, \text{ReLU(input)} &= \text{input} \end{aligned}$$

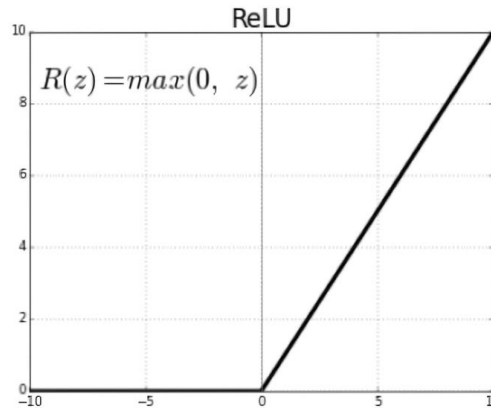


Fig 10: ReLU function. (Source: <https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2>)

There are four such combinations followed by a flatten and a dense layer with ‘Softmax’ activation. To avoid the problem of overfitting, Dropout regularization is used. It is a regularization technique which excludes input and recurrent connections to LSTM/CNN units probabilistically from weight and activation and updates during network training. As a result, overfitting is decreased and model performance is enhanced. The architecture of CNN used in this project is as described in Table 2 below:

Layers	Units	Activation	Parameters
Convolutional Layer	256	ReLU	1536
Convolutional Layer	128	ReLU	163968
Dropout	0.1		
Max Pooling Layer	8		
Convolutional Layer	128	ReLU	82048
Convolutional Layer	128	ReLU	82048
Flatten Layer	7		
Dense Layer	7	Softmax	6279

Table 2: Architecture of CNN model

3.4.2 Multi-Layer Perceptron:

According to Abirami and Chitra, (2020), Multilayer perceptron (MLP) and a basic feed forward neural network is supplementary of one another. The input layer, hidden layer, and output layer are the three distinct kinds of layers that constitute any basic MLP model. Except for the input nodes, each node in the MLP is a neuron that employs a nonlinear activation function such as sigmoid to allow the model to learn and solve a far more complex classification problem. This is to help solve the problem of non-linearity of the data. The input layer is where the input signal for processing is received. The output layer manages the required tasks, like categorization and prediction. Between

the input and output layers are any number of hidden layers that make up the MLP's actual computational engine. Given below in Figure 11 is a basic MLP architecture consisting of the three layers – Input, Hidden, Output.

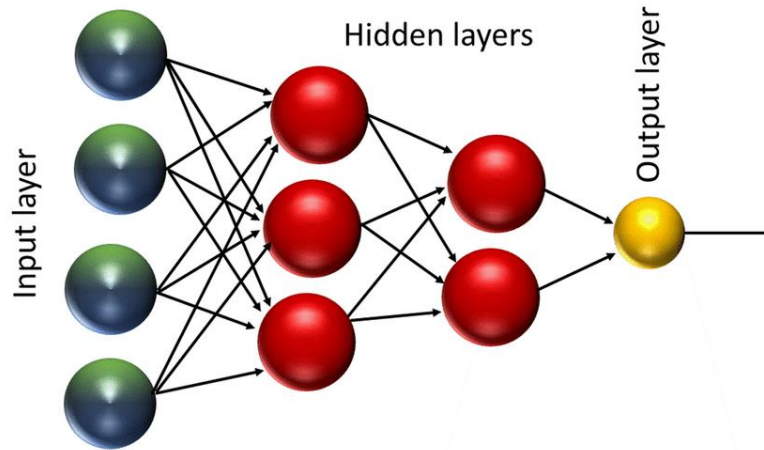


Fig 11: Representation of MLP model. (Source: https://www.researchgate.net/figure/Multilayer-perceptron-showing-input-hidden-and-output-layers-and-nodes-with-feedforward_fig2_342723126)

The data moves from the input layer to the output layer of an MLP network in the manner reminiscent of a feed-forward network. To train the neurons present in all the layers of the multi-layer perceptron, the most common technique used is the back propagation learning algorithm. According to Abirami and Chitra, (2020), multi-layer perceptron networks are created to approximate any continuous function and can address issues that cannot be resolved linearly. MLPs are hence quite useful for classification problems. One such application is recognition and classification of emotions in speech/features.

For this project, an MLP classifier, with hidden layers of sizes 250 and 150 neurons, was built with an iteration of 400.

3.4.3 Long Short-Term Memory:

A subgroup of neural networks is the recurrent neural networks (RNN) in which the results of previous steps are used as inputs for subsequent steps. Typical neural networks feature inputs and outputs that are not reliant on one another. However, in circumstances when it is important to anticipate the following data in a series, such as guessing the next word in a phrase, it is necessary to remember the preceding information, such as the previous words. RNN was created as a solution and it used a "Hidden Layer" to fix this issue of remembrance. Thus, time series data and sequential data learning is best accomplished with recurrent neural networks (RNN). But, RNN models have a vanishing gradient issue that grows worse with training sequence length, despite the fact that they are good at learning temporal correlations. Thus, LSTM (Long

Short-Term Memory) RNNs were invented by (Sepp and Jurgen, 1997) and (Hochreiter et al., 2001) to address this issue. According to Chen and Jin, (2015), in LSTMs, information is stored in memory cells so that it can benefit from the data's long-range dependencies.

LSTM features feedback connections as opposed to typical feedforward neural network. It can analyze whole data sequences, for example, speech (only audio) or a video (both audio and visuals) in addition to single data points like photos. A typical LSTM network consists of multiple memory cells and includes states or information that is transmitted back and forth. The hidden state and the cell state are transmitted to the memory cell. Information is stored in memory blocks, which use three major gates —forget, input, and output gate. These are used to manipulate this memory. An illustration of a typical LSTM architecture and its rolled-out version at different steps is shown below in Figure 12.

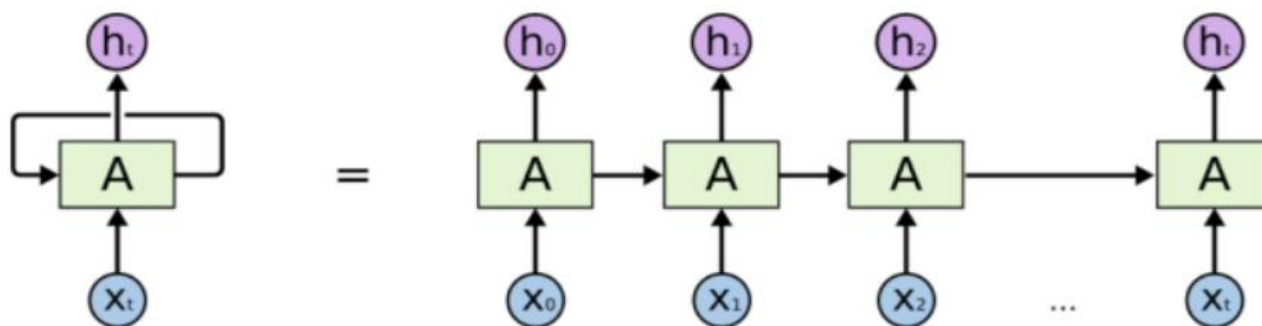


Fig 12: RNN Model. Source: https://www.researchgate.net/figure/Rolled-out-RNN-at-different-time-steps-Long-Short-Term-Memory-8-is-one-of-the-popular_fig1_341151347

Forget gates: Recurrent gates referred to as "forget gates" are typically added to LSTM. Looking at an example, taking a sentence:

‘Marley is a good man. Riles is a good man too.’

The forget gate in this case detects the possibility of a context shift in the next line as soon as it encounters the ‘full stop’ following the word "man." As a result, the topic of the phrase is overlooked, and its original location is freed. And when the next sentence begins talking about "Riles," this subject's place is assigned to him. The forget gate causes this process of forgetting the subject (**Srivastava, 2017**).

Input Gate: The input gate is in responsible of updating the cell's state with information. Here, additional information in the aforementioned example would be that both Marley and Riles are good people (**Srivastava, 2017**).

Output Gate: The output gate is liable of extracting pertinent information from the present state of the cell and projecting it as output. In other terms, predicting the next word of a sentence based on the states received from input gate and the information stored in the memory (Srivastava, 2017).

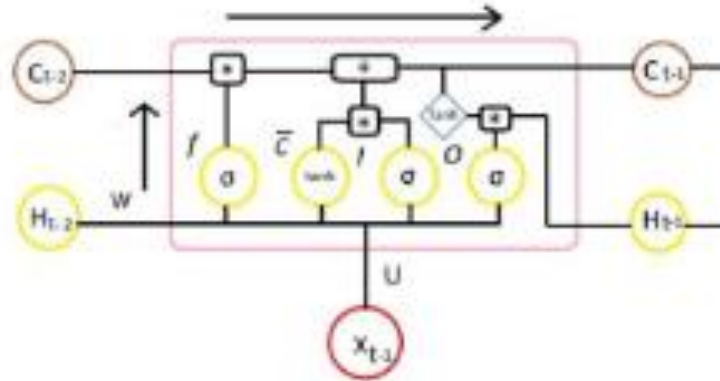


Fig 13: LSTM Model with Input, Output and Forget gates

Figure 13 shows the LSTM model containing the output gate (O), Input gate (I) and forget gate (F). The two hidden states are depicted by (H) and candidate layer is shown by (C).

Backpropagated errors can be avoided from exploding or vanishing using LSTM. Instead, errors might go backwards through an endless number of simulated layers that are dispersed in space. In other sense, according to Sai, et al., (2022), tasks requiring memories of occasions that occurred thousands or even millions of discrete time steps previously can be learned using LSTM. Employing RNN-LSTM has the bigger benefit of performing well with large datasets. For emotion detection, the interdependence of each part of the audio speech with the previous one should be taken into account. So, in this case LSTM should ideally give a better performance.

In this project, the best version of the LSTM model was built with a simple RNN architecture is built with the parameters as shown below in Table 3:

Layers	Units	Activation	Parameters
LSTM	200	Tanh (Default)	161600
Dropout	0.5		
Dense	512	ReLU	102912
Dense	512	ReLU	262656
Dense	7	Softmax	3591

Table 3: Architecture of RNN model

3.4.4 Support Vector Machines:

Support Vector Machine (SVM) is a well-known supervised machine learning method which is utilized in both regression and classification algorithms. However, classification algorithms tend to favor it. It essentially creates a hyperplane in n- or multidimensional space that divides various target groups. The fundamental goal of the SVM is to establish the optimum decision border with the greatest amount of margin between two or more classes so that the fresh data points can be accurately classified. (Awasthi, 2020).

In machine learning algorithms, one of the best classifiers with a considerably optimal margin are Support Vector Machines. SVMs are extensively used in numerous research papers that focus on audio/speech emotion recognition, example, (Hu, Xu and Wu, (2007), Chavhan, Dhore and Yesaware, (2010), Shen, Changjun and Chen, (2011)). According to G.S. et al., (2016), in comparison to other classifiers/models, it can do remarkably well in terms of classification, especially with little training data. According to (Awasthi, 2020), SVMs are have a better ability of handling classification and regression methods on both linear and non-linear data. The models usually are more accurate when compared to other algorithms due to their ability to handle small (typically tens to many thousands of features) and complex datasets. Another benefit of it is it can find complex relationships amongst the provided dataset without preprocessing it with plenty of transformations. This is handled by Kernels. Kernels help a lot when dealing with complex datasets. They receive the data as input, which they then transform into the necessary form. Typically used kernels are:

- The linear kernel is used to partition data linearly. Because it works well with big datasets, this kernel is often chosen for text classification models.

$$\text{Formula: } F(X, X_j) = \sum (X \cdot X_j)$$

- Sigmoid kernel graphs into a sigmoid when classifying data and is mostly preferred for neural networks. This kernel closely resembles a two-layer perceptron of a neural network and functions as an activation function for neurons.

$$\text{Formula: } F(X, X_j) = \tanh(\alpha x y + c)$$

- The Gaussian Radial Basis Function (RBF) kernel is a type of gaussian kernel, which projects the higher dimensional data and then searches for a linear separation plane for the data.

$$\text{Formula: } F(X, X_j) = \exp(-\gamma * ||X - X_j||^2)$$

- Polynomial kernels are used preferably for data which has been normalized. This kernel is not often used as it does not give the most reliable results when compared to other kernels.

$$\text{Formula: } F(X, X_j) = (X \cdot X_j + 1)^d$$

The below Figure 14 depicts the graphical representations of SVM classifications using different kernels for some dummy/example data.

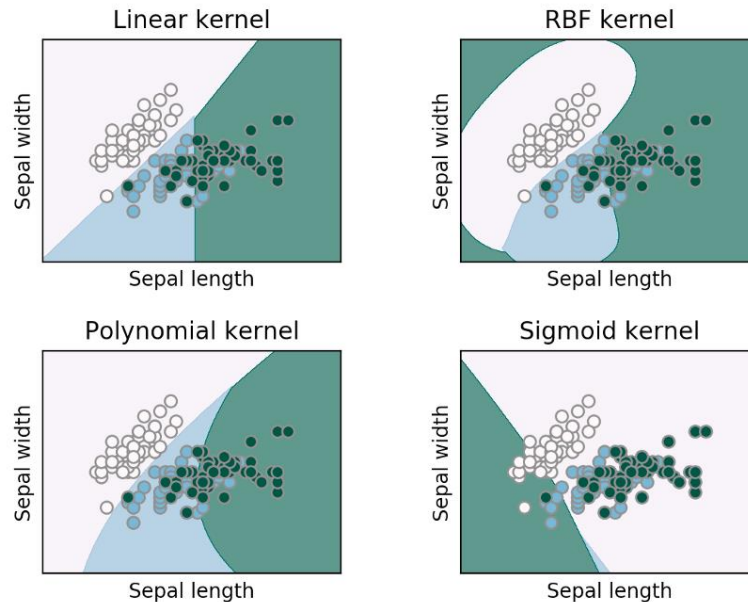


Fig 14: Visual SVM kernels Example. (Source: <https://towardsdatascience.com/multiclass-classification-with-support-vector-machines-svm-kernel-trick-kernel-functions-f9d5377d6f02>)

For this project, SVM model was built with RBF kernel which gave the highest accuracy amongst the other kernels.

4. Results

The table below displays the comparative outcomes for each model. This table displays the accuracy, training time, testing time and predictive quality of the best versions of each model trained in the project.

Model	Accuracy (TP+TN)/ (TP+FN+TN+FP)	Training Time	Test time	Predictive quality
CNN	84.47%	82mins	7.48 sec	Very good
MLP	86.29%	6.56 mins	0.19 sec	Very good
RNN-LSTM	98.14%	63.38 mins	2.23 sec	Excellent
SVM	72.80%	6 mins	~ 1 min	Average

Table 4: Results of each model

The predictive quality was measured based on the accuracy score. Approximately 17,900 (30%) feature- extracted data files were tested for CNN and MLP and 14400 (20%) data files for RNN and SVM. As highlighted, RNN yielded the best accuracy and had excellent predictive quality. The training and test times were measured (as start time – current time) from the time.time() command in the program.

CNN Model: The dataset was reshaped to make it compatible with CNN model. The RMSprop optimizer was used with an adaptive learning rate starting from 0.01 decreasing by a factor of 0.4 to the lowest of 0.0000001. 50 epochs were used to train the CNN model having batch size of 256. CNN took approximately 82 mins to train with training loss deprecating from 3.00 to 0.19 and testing loss from 1.31 to 0.60. While the training accuracy was 93.25%, the validation accuracy could only reach 84.47%. Below are the graphs representing training and testing loss and accuracy.

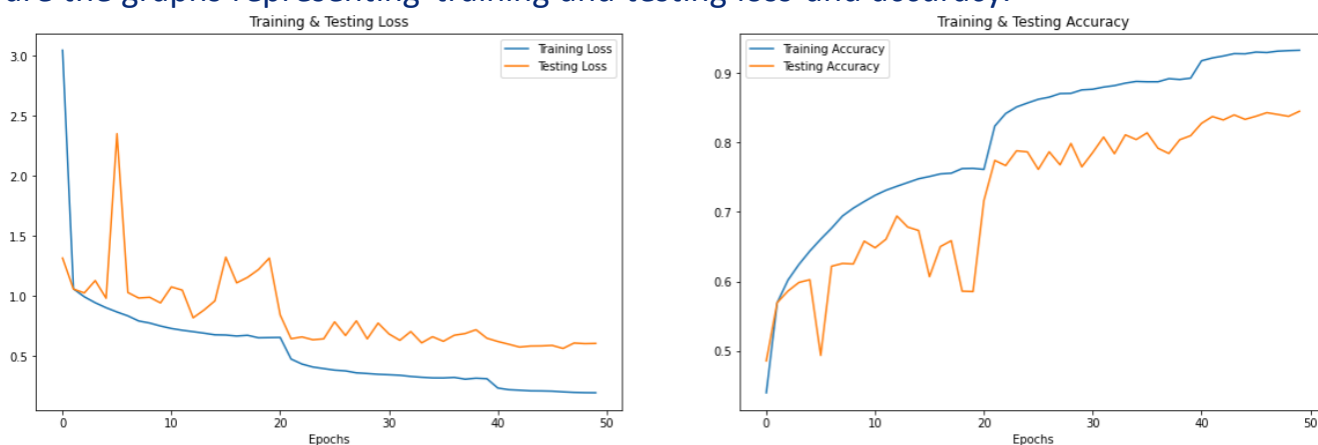


Fig 15: Plot of Accuracy and Loss of CNN model on training and validation data

At first, different models with varied parameters and layers were tried out but the aforementioned model proved to give the best results. At first, just three Conv layers with max pooling were tried out which gave an accuracy of approx. 76%. Then the

parameters were adjusted ranging from 64 to 256 each resulting in different accuracies and training times. Also, the model was trained for more epochs of 60 and 75, but the losses did not improve and the model tended to overfit the data. Different optimizers were also tried out including Adam, Nadam, RMSprop, and SVG. but the best optimizer was RMSprop for this classification problem.

MLP Model: The best version of MLP classifier was built with two hidden layers of 250 and 150 neurons respectively. It had an adaptive learning rate, and alpha value of 0.06. An alpha value is used to tackle overfitting/underfitting problems by restricting the weight size. Increasing alpha fixes high variance while decreasing its value fixes high bias. Thus, an ideal value is to be calibrated to achieve reliant results. The epsilon of 0.1×10^{-8} (used for stability of optimizer Adam) was used. A batch size of 256 was employed. The MLP took approximately 6.56 mins to train and 0.19 seconds to test giving an overall accuracy of 86.29%. Other versions were built with different combinations of hidden layers and batch sizes which affected the accuracy and training time of the models. Some of the models worth mentioning for comparison are:

Hidden layers	Batch size	Training time	Accuracy
(300,)	256	~ 9 mins	74.85%
(350,)	256	~ 14.52 mins	77.50%
(350,)	128	~ 13 mins	75.23%
(400,)	256	~ 14 mins	76.55%
(300, 100)	256	~ 7.5 mins	79.56%
(250, 150) – best version	256	~ 6.6 mins	86.29%

Table 5: Different versions of MLP models built in this project

As observed from Table 5, increasing the parameters did increase the accuracy but heavily affected the training time. Increasing layers resulted in lesser training time and increase in accuracy.

SVM Model: RBF (Gaussian Radial Basis Function) kernel was used to train the most effective SVM model. The approximate training time taken by the model was 6 mins and it took a rather long time to test of approximately 73 seconds. The overall accuracy achieved was 72.8% with the overall predictive quality being average to low. At first, different kernels were used for training including linear, polynomial and sigmoid. While polynomial kernel gave around 70% accuracy with a training time of approximately 7 minutes, giving second best results, linear kernel and sigmoid kernel gave even worse results of about 56% and 24% of accuracy respectively with an even greater training time of about 17 mins and 7.5 mins respectively.

RNN Model: The RNN model was trained with the loss metric of 'binary cross-entropy' and 'Nadam' optimizer. An adaptive learning rate starting from 0.01 decreasing by a factor of 0.4 to the lowest of 0.000001 was applied. RNN was trained with a batch size of 128 across 60 epochs. This model took approximately 64 mins to train and gave an accuracy of 98.13%. The validation loss captured during the training of this model reduced from 0.27 to 0.13 while the validation increased accuracy from 0.50 to 0.91. Below are the graphs representing training and testing loss and accuracy.

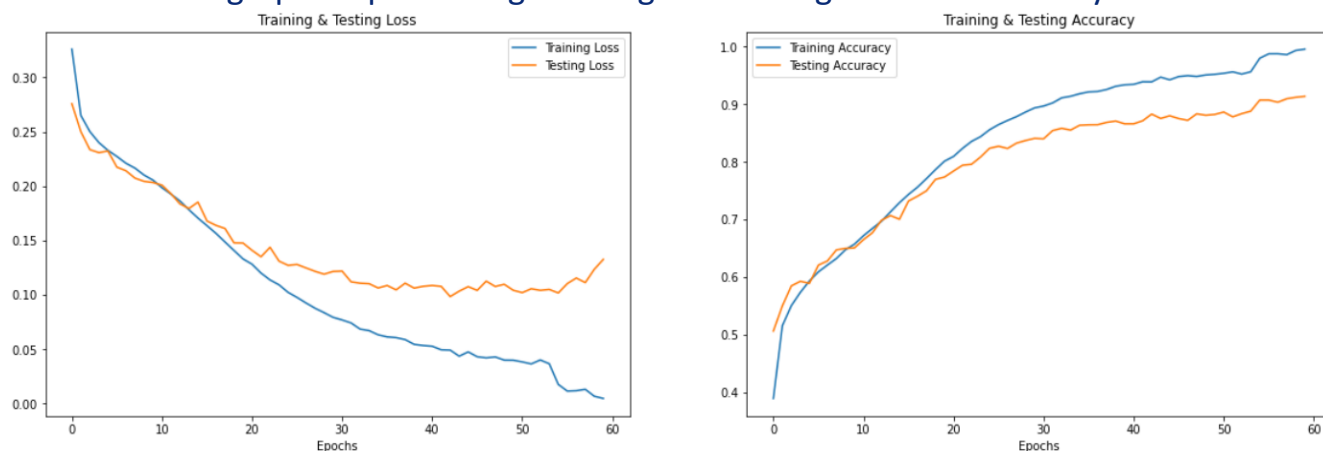


Fig 16: Plot of Accuracy and Loss of RNN model on training and validation data

Again, different parameters with a varied number of layers were tried for this classifier. The model with same number of layers but with just 30 epochs gave an accuracy of 68%, so the epochs were increased to 50, the accuracy improved to 72.5%. Another approach of scaling the features before they were trained was tried but the results were not very promising and gave lesser accuracy than before. Optimizers like Nadam, RMSprop and Adam were tried and 'Nadam' proved to be the best optimizer. The model could reach an accuracy of 78.9% with learning rate of 0.01 even when trained for 100 epochs. Then, the learning rate was made adaptive and epochs were increased to 100. This model gave the second-best results. To experiment further, the training data was increased to 80% from 70%, decreasing the test dataset. This did not make much difference to the model performance. Then, a second layer of LSTM was added with same number of parameters (100). Though it did increase the accuracy, due to the model complexity, the training time increased by 50 mins and test time increased by 6 secs making it unfit for real-time deployment. Finally, only one layer was kept and parameters were increased from 100 to 150 to 200 and epochs were decreased to 60. The best model was achieved by one LSTM layer of 200 parameters with two dense layers.

The overall effectiveness of the models in identifying certain emotions is examined using the confusion matrix. It is used to determine how well each model correlates with each

emotion. Because they give a more realistic representation of a model's performance, confusion matrices are the most common. In a regular accuracy measure of classification, the information of incorrectly classified cases is not reported. A confusion matrix provides information of both rightly and erroneously classified cases reported giving us a broader understanding of the model's performance. To understand the confusion matrix, below are a brief definition of terms used in the matrix.

Precision: It is an indicator of the level of accuracy attained in true prediction. In plain English, it informs us of the proportion of real positive predictions to all positive predictions. Ideal case value of precision is 1 (Suresh, 2020).

Recall: or sensitivity is described as the percentage of positive events that were correctly identified out of all positive events. Or, which positive instances, if any, were correctly predicted out of all of them. Recall ought to be high (ideally 1) (Suresh, 2020).

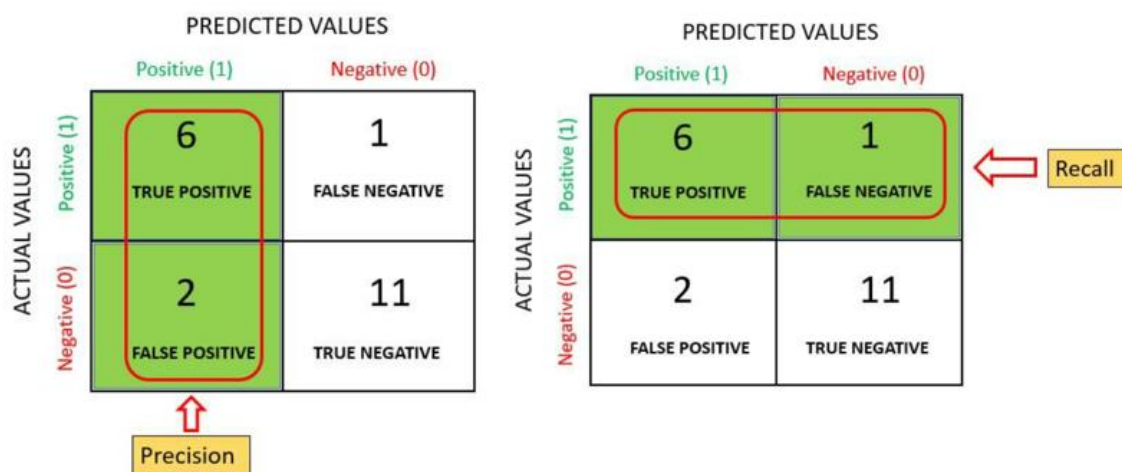


Fig 17: Precision and recall. (Source: <https://medium.com/analytics-vidhya/what-is-a-confusion-matrix-d1c0f8feda5>)

F1 score: essentially achieves a balance between a classifier model's precision and recall parameters. It is directly proportional to both recall and precision i.e., if precision and recall are low, then F1 score is also low (Suresh, 2020).

$$\text{F1-Score} = 2 * \frac{(\text{Recall} * \text{Precision})}{(\text{Recall} + \text{Precision})}$$

A Classification Report along with the Heatmap for each model: To further analyze the performance of each model in terms of performance level of detection of each emotion, the heatmaps and classification reports are shown below for each model in figures 18,19,20,21.

	precision	recall	f1-score	support
angry	0.80	0.84	0.82	2314
disgust	0.64	0.65	0.65	2353
fear	0.79	0.61	0.69	2365
happy	0.74	0.68	0.71	2307
neutral	0.69	0.70	0.70	2125
sad	0.66	0.81	0.73	2299
surprise	1.00	1.00	1.00	576
accuracy			0.73	14339
macro avg	0.76	0.76	0.76	14339
weighted avg	0.73	0.73	0.73	14339

----accuracy score 72.80842457633028 ----

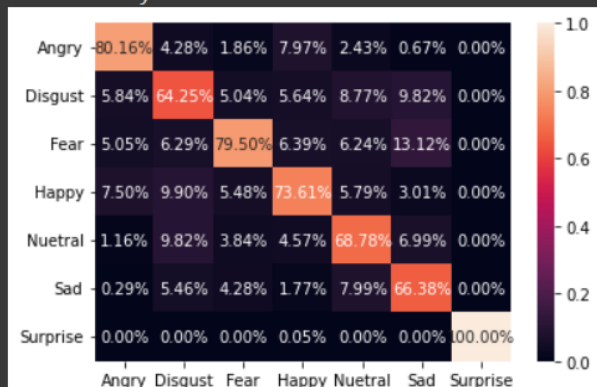


Fig 18: Classification report and heatmap of SVM

	precision	recall	f1-score	support
angry	0.93	0.93	0.93	2894
disgust	0.92	0.79	0.85	2933
fear	0.64	0.94	0.76	2866
happy	0.91	0.84	0.87	2975
neutral	0.92	0.86	0.89	2690
sad	0.95	0.81	0.87	2887
surprise	1.00	0.98	0.99	679
accuracy			0.86	17924
macro avg	0.90	0.88	0.88	17924
weighted avg	0.88	0.86	0.87	17924

----accuracy score 86.28654318232537 ----

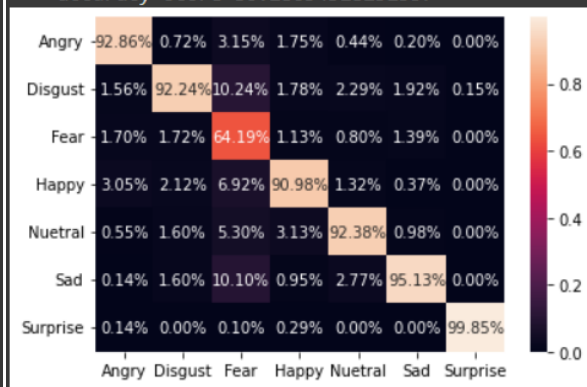


Fig 19: Classification report and heatmap of MLP

	precision	recall	f1-score	support
angry	0.98	0.99	0.99	10496
disgust	0.98	0.98	0.98	10579
fear	0.98	0.98	0.98	10548
happy	0.98	0.98	0.98	10498
neutral	0.98	0.98	0.98	9361
sad	0.98	0.98	0.98	10520
surprise	1.00	1.00	1.00	2523
accuracy			0.98	64525
macro avg	0.98	0.98	0.98	64525
weighted avg	0.98	0.98	0.98	64525

----accuracy score 98.13560635412631 ----

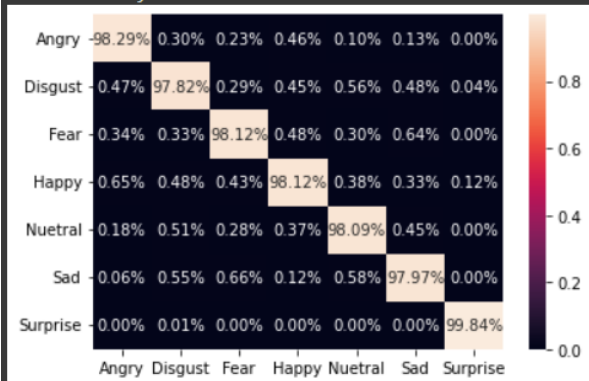


Fig 20: Classification report and heatmap of RNN

	precision	recall	f1-score	support
angry	0.89	0.90	0.90	2894
disgust	0.80	0.80	0.80	2933
fear	0.84	0.83	0.83	2866
happy	0.85	0.82	0.84	2975
neutral	0.82	0.83	0.82	2690
sad	0.83	0.86	0.84	2887
surprise	0.99	0.99	0.99	679
accuracy			0.84	17924
macro avg	0.86	0.86	0.86	17924
weighted avg	0.84	0.84	0.84	17924

----accuracy score 84.47333184557019 ----

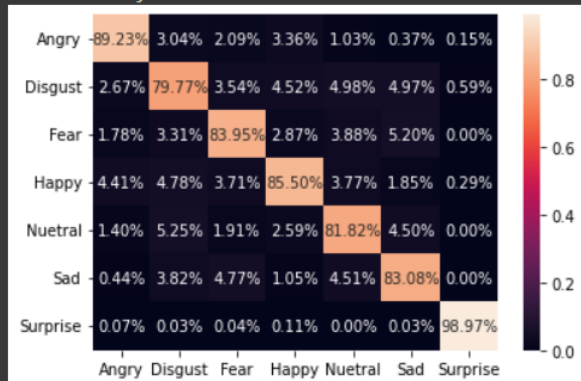


Fig 21: Classification report and heatmap of CNN

SVM Model: From Figure 18, the precision with which the "surprise" and "anger" emotions were detected for the SVM model was equivalent to or higher than 80%, suggesting that these emotions were detected with a higher degree of precision than other emotions. The least precise emotions were "sad" and "disgust," which suggests that the model failed to correctly identify these feelings. The recall rate varied from 60% to 99% for all emotions, suggesting a moderate level of success for the model in identifying false negatives. The overall F1 score too had similar range of 65% to 100% for the emotions implying that though the model's capability of identifying few emotions was good, it could not distinguish between 'disgust' and 'fear'. The heatmap of the SVM model shows a rather detailed picture of each emotion classification whether correctly detected or falsely classified. The correlation of each emotion with itself and with the other emotions as well is depicted. For example, while emotion 'angry' was correlated, rather detected with 80.16% accuracy, it was falsely detected or correlated as 'happy' 7.50% of the times, 'disgust' as 5.84% of the times etc.

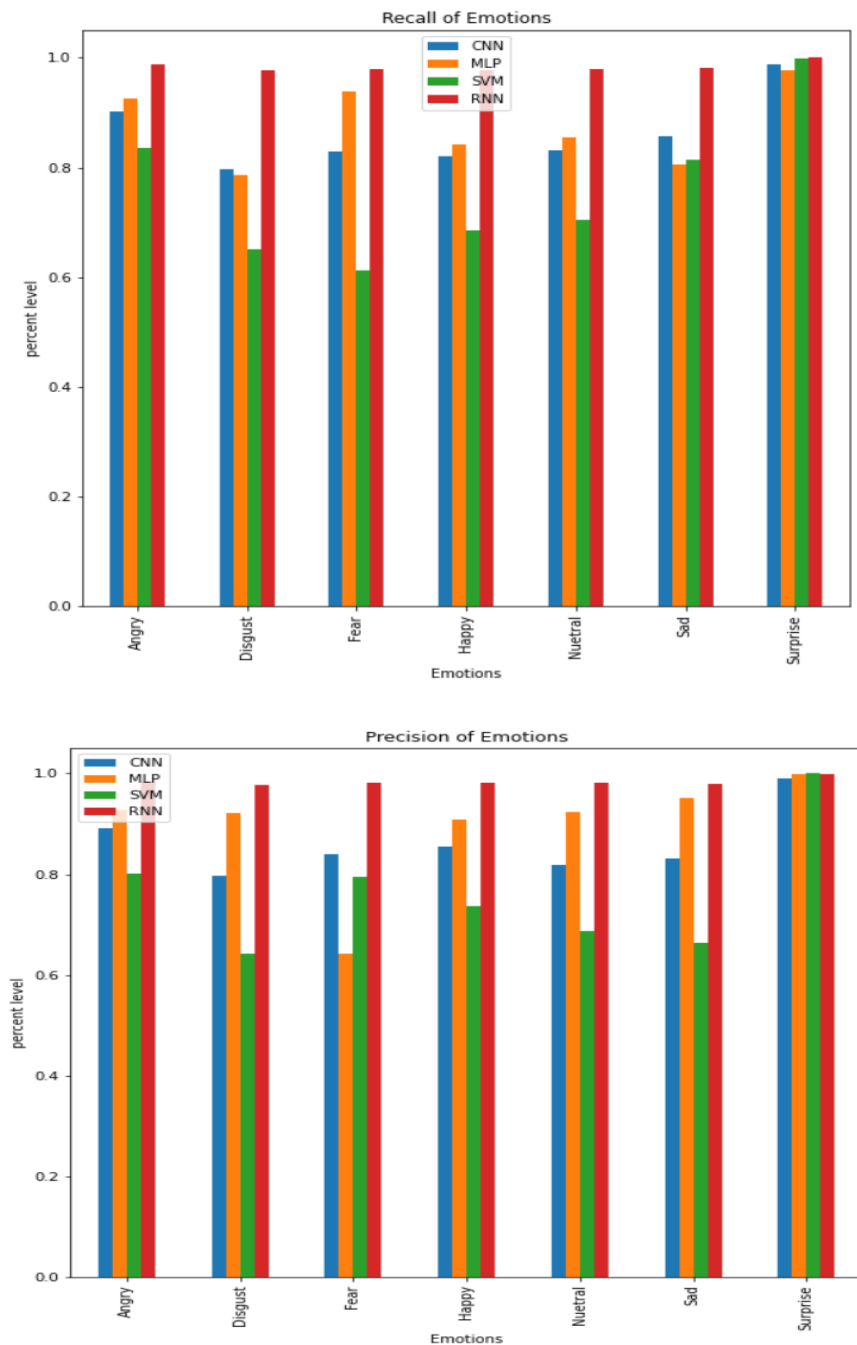
MLP Model: For the MLP model as shown in Figure 19, 'angry', 'happy', 'neutral', 'disgust', 'sad' and 'surprise' emotions performed extraordinarily well, with the model's accuracy in identifying these being more than 90%. A good recall rate was also observed for these emotions, the lowest being 79% for 'disgust' emotion which was far better than SVM model. F1 score was the highest for 'surprise' recorded at 99% implying that model almost always classified this emotion accurately. The aggregate F1 score remained higher than 86 percent, suggesting that this model performed well overall. A more graphical representation of correlation of each emotion is shown in the heatmap of MLP model below.

The RNN model: in Figure 20, which was the most accurate model, had a precision level of above 97% for all the emotions, with "surprise," "fear," "happy" and "anger" having the best precision of more than 98%. All of the emotions had recall rates above 98%, which speaks to the model's accuracy in identifying false negatives. The total F1 score was above 98% for all emotions, demonstrating the model's strong capacity to recognize and distinguish emotions with greater accuracy. As observed, all the emotions had the same levels of precision and recall rate which speaks about the model's stability and consistency amongst all emotions.

CNN Model: All emotions for the CNN model were within the 80–90% precision range as seen in Figure 21, preserving no biases against any emotion. Only one emotion, 'surprise' was 99 percent accurately represented. In comparison to previous models, a consistent recall rate of between 80 and 90 percent was also found, with "surprise" being the only

emotion that showed preference over other emotions. This resulted in a pretty balanced F1-score of all the emotions. Although, MLP had higher overall accuracies, CNN, as LSTM, had an advantage of being consistent across all emotions whereas SVM and MLP had few emotions with very less precision. Heatmap of CNN is as shown below, giving a more detailed analysis of each emotion and its correlation.

To further understand the performance of models on emotion level, the graphs were plotted for precision, F1 Score and recall of each emotion detected by the four models below in Figure 22:



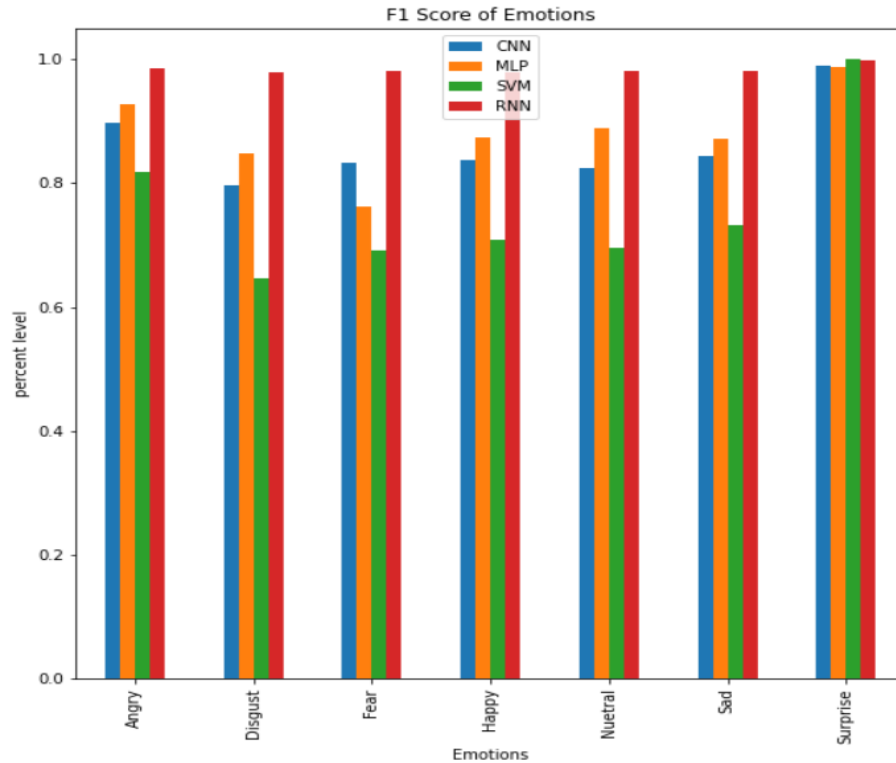


Fig 22: Statistics of Each Emotion

From the confusion matrices in Figures 18,19,20 and 21 and the graphs in Figure 22, all the emotions were classified with very good accuracy in all the models, but 'surprise' emotion had the F1-score of almost 100% in every model. It was observed that the models may have overfitted this emotion as 'surprise' emotion had very limited data in comparison of other emotions. Emotions 'Angry' and 'Happy' was the most precise emotions with good recall rate as well. Emotions 'Disgust' and 'Fear' were the emotions which had less overall precision in the models, inferring that these emotions were not classified with highest precision. The emotions 'Sad' and 'Neutral' were moderate in terms of precision and recall rate.

5. Discussion of Results

5.1 Comparison of models:

From the results, it is clear that the RNN provided the maximum accuracy of 98% but required roughly 64 minutes to train and 3 seconds to test, whereas the MLP model provided an accuracy of 86% with a testing time of under a second, providing the best accuracy to time trade-off implying that MLP could be more usable in real-time scenario. Even though SVM trained the quickest in around 6 mins, it only could reach 72.8% which did not quite meet the proper accuracy level and tested in around one minute, which questions the credibility of the model, again inferring that the model was not quite ideal in a real-world scenario. 84% accuracy was provided by CNN, however the testing of it took 7.48 seconds, which is again not optimal in a real-time scenario. As seen from the performance of detection of each emotion, CNN and LSTM were consistent across all the emotions in terms of precision of each emotion, whereas MLP and SVM had lesser precision for few of the emotions.

5.2 Comparison with Other Papers:

The SVM Model in Kerkeni et al., (2018) attained an accuracy of 77.63% on Spanish dataset and 63.30% on berlin dataset whereas our model achieved 72.8% of accuracy. The accuracies of the models were differed by the datasets taken. Taking a closer look at the datasets, the Spanish dataset has audio recordings of 7 basic emotions from only two professionals – one male and one female which implies that the models are more likely to have been overfitted due to limited pitch and tone data hence giving higher accuracy. The berlin dataset has audio recordings of 10 different professionals giving a lot more data in terms of tone and pitch variations. This would imply that the models built were robust and were immune to overfitting resulting in less accuracy when trained on that dataset. For the SVM model built in this project, the dataset had a lot of variation in tone and pitch as it had recordings of 93 professional actors and yet managed to achieve a sufficient accuracy of 72.9%. For RNN model, Kerkeni et al., (2018) achieved an accuracy of 90.05% on Spanish dataset and 69.55% on berlin dataset while the RNN model in this project achieved an accuracy of 98% again implying that the datasets involved were affecting the accuracy and precision of the models built. Another defining factor was the feature extraction technique involved. Kerkeni et al., (2018) integrated MS (Modulation Spectral) and MFCC features in addition to employing MFCCs and MS features separately, however this project solely used MFCC feature extraction. As can

be seen from their research, combining MFCCs and MS features produced superior results to using just MFCCs or MS alone.

Mao et al., (2014) talked about just CNN model and its classification accuracy when trained on four datasets. The CNN models built by them achieved an accuracy of 73.7% for SAVEE (Surrey Audio-Visual Expressed Emotion Database) data, 85.2% for EMODB (Berlin dataset), 79.9% for DES (Danish Emotional Speech database) and 78.3% for MES (Mandarin Emotional Speech). The features were extracted with their own algorithm. Comparing the highest and the lowest accuracies for the datasets (highest for EMODB and lowest for SAVEE), EMODB had recordings of 10 actors while SAVEE had just 4 English actors. One of the positive points of this paper was it explored the language/culture barrier of speech and its underlying emotion. In comparison to this project, CNN model had an accuracy of 84.47% performing better than most of the models in Mao et al., (2014) as it used MFCC feature extraction technique which means that the CNN model of this project was trained on spectral features (MFCCs) whereas the other CNN models of Mao et al., (2014) were trained on prosodic features (pitch, tone etc.). As seen from paper Shen, Changjun and Chen, (2011), using the same dataset, model achieved more accuracy when trained with spectral features than prosodic features.

Convolutional Neural Network (CNN) was also employed in the paper Issa, Fatih Demirci, and Yazici, (2020) to construct SER. The datasets used in this study differed from those used in Mao et al., (2014). On the Berlin dataset, this article was able to reach an accuracy of 86.1%, which was greater than the accuracy 85.2% obtained by Mao et al., (2014). Although the dataset was the same, the different accuracy levels were caused by the feature extraction methods used in these articles. While Issa, Fatih Demirci, and Yazici, (2020) used MFCCs for spectral feature extraction, Mao et al., (2014) employed their own feature extraction technique which involved prosodic feature extraction. While Issa, Fatih Demirci, and Yazici, (2020) focused on building the CNN model of 6 CNN layers, Mao et al., (2014) focused on the feature extraction technique to make the SER system more robust in nature.

While CNN model developed on the dataset RAVDESS (Ryerson Audio-Visual Database of Emotional Speech and Song) employed in paper Issa, Fatih Demirci, and Yazici, (2020) showed an accuracy of only 71.61 percent, RNN technique used to develop the SER system in Sai, et al., (2022) on the same RAVDESS dataset showed an accuracy around 95%, suggesting that RNN could be a better deep learning algorithm than CNN for SER systems. Despite the fact that the RNN in Sai, et al., (2022) reached an accuracy of 95%, the RNN in this project achieved roughly 98% as a result of a far more robust and varied

dataset and more data than RAVDESS (RAVDESS contains only 1440 files, whereas this project had a dataset of 10,242 files). Another differentiating factor was that the features were extracted using Short Time Fourier Transform (STFT) by Sai, et al., (2022), whereas this project used MFCCs indicating that the feature extraction method also has a significant impact on the variation in model accuracy.

5.3 Applying Models to some of the Applications:

For any invention/development, its quality and value can only be justified when applied to real-world scenarios. Hence, taking a deeper look at the usability of these models on some of the popular applications is the next natural step. For call centers and detection of fraud, since the detection of emotions through the voices of the customers has to be instantaneous, a model with higher testing speed with right accuracy should be deployed, hence the MLP or RNN models would make a better fit. For Medical studies and NGOs, the accuracy of the model should be the utmost priority. Since feedback is not an immediate requirement, the RNN model would be the right fit. Also, accuracy of each emotion individually is a matter of priority. Hence, CNN can also be deployed in this field. For analyzing the emotion levels of the politicians to gauge their connection with the masses, it is crucial to have an exceedingly high accuracy and a good testing time which can be achieved by RNN and MLP models. Interfacing of AI, including Siri/Alexa requires simultaneous emotion detection with each sentence spoken to it. Though accuracy is crucial, it is not an extremely critical factor for these interfaces. Hence, deploying either RNN, MLP or CNN model would give us superior results.

5.4 Improvements of Models:

- Since the SVM model had the least accuracy and poor performance metrics, the model could be improved by trying out other feature extraction techniques like MS or by using prosodic features instead of spectral features.
- During the training of LSTM and CNN models, it was observed that those models took a lot of time to train. To improve on that, model optimization techniques like 'prefetch', 'cache', 'batch', etc. could be employed.
- Since the test data is taken from the same dataset as training data, models may not be quite robust in nature and tend to overfit. Instead, test data from other datasets can be taken to gauge the right test accuracy and other metrics.
- Individual voice recordings can be utilized as input to evaluate the testing time of these models to assure their speed in a real-time environment, and a user interface can be constructed and interfaced with the backend models.

-
- Experimenting with the division of percentage of train and test data can be tried out as opposed to the standard 70%-30% to improve training of the models.
 - One of the observations with the dataset in this project was that the emotion 'Surprise' had very less data and hence the models in this project overfitted this emotion. More data can be increased in order to get the right training of these models.
 - Increasing the emotional range of dataset would help the models predict more emotions as opposed to just the seven emotions predicted currently.

5.5 Limitations:

One of the major limitations of this project was that the datasets taken had the voice recordings being acted out by professional actors in professional environment like silent rooms or recording studios, whereas in real-time applications, the emotions must be detected for people that have a wide range of emotional aptitude with complex or mixed emotions and the SER systems should also be accustomed to real-world environment which may contain noise and other characteristics. Another aspect to be considered is the effects of language and culture on SER. For instance, there may be variances in the way emotions are expressed in speech across different languages. According to Akçay and Oğuz, (2020), there are many ongoing studies for development of cross-language SER models. However, with the current models and techniques, the results are not as promising as required. Another aspect is to attain the required accuracy. In order to increase emotion recognition accuracy and reduce the computational intricacy of the entire model, there are numerous obstacles in the literature and research of SER. Due to these challenges, a right balance of accuracy vs model complexity is yet to be originated. Generalization is also one of the challenges that SER faces, in terms of the models giving the same accuracy and testing time with every dataset the model is being trained and tested with. Also, the present SER models detect just basic emotions, mostly the common emotions of sad, happy, anger, disgust, neutrality, and fear. But, in real life scenario a human subject may emote complicated or mixed emotions as well. With the models built currently, those emotions may either be undetected or classified incorrectly.

6. Conclusion

To summarize, a Speech Emption Recognition system will have an enormous impact on human-Artificial Intelligence (AI) interaction with an extensive range of applications. A breakthrough to get the right model-accuracy model to be able to deploy the system in real-world applications is still a topic for research. According to Kerkeni et al., (2020), There are still many unknowns regarding the ideal method for categorizing emotions. The rate at which emotions can be detected varies depending on the combination of emotional traits. What characteristics affect the recognition of emotion in speech is still a subject of discussion among scholars. In this project, the SER system is developed on a dataset merged with two known datasets. The combined dataset is used to train and test the four models for comparison. For this project, the best model was MLP classifier as it could nearly attain the right accuracy-time balance giving over 86% accuracy and approximately 7 mins for training time. RNN proved to be the best in terms of accuracy but had the disadvantage of training time of over 63 mins and test time of over 2 seconds. Though the robustness of this model is still open to research, the basic emotions were correctly classified. As for the limitations, generalization is a major concern across the research and real-time applications. Also, classification of real-time mixed or complex emotions is a challenge yet to be solved.

Future Scope:

- A possibility is to make it a more generalized version of models to be able to overcome overfitting. This would also help in maintaining the accuracy level given the kind of raw, real-time data generally available in organizations.
- The exploration of detection of changing emotions during a brief period using the SER system can also be useful for real-world applications.
- To overcome the limitations of getting real-time emotions for training the models, the possibility of getting real-life data from established organizations like call-centers, NGOs etc. to train data can be explored.
- Another possibility is to research/build models which can detect complicated emotions or mixed emotions.
- At present many researchers tend to develop new models based on supervised learning techniques, further research may lead the researchers to explore the possibility of using unsupervised training techniques.
- Exploration of combination facial expressions along with the speech to enhance the recognition of emotions which will also help in interfacing AI robots.

-
- Another possibility of using a combination of sporadic-spectral features or combination of MFCC-MS or MFCC-LLD features can be explored to improve the SER system's robustness.
 - Combination of models, for example CNN+LSTM, maybe explored as it may prove to be a more a far superior algorithm in building SER.

7. References

Abirami, S. and Chitra, P., 2020. Energy-efficient edge based real-time healthcare support system. *Advances in Computers*, pp.339-368.

Akçay, M. and Oğuz, K., 2020. Speech emotion recognition: Emotional models, databases, features, preprocessing methods, supporting modalities, and classifiers. *Speech Communication*, 116, pp.56-76.

Anderson, M., 2022. The Future of Emotion Recognition in Machine Learning. [online] Iflexion. Available at: <<https://www.iflexion.com/blog/emotion-recognition-software>> [Accessed 5 July 2022].

Awasthi, S., 2020. SEVEN MOST POPULAR SVM KERNELS. [Online] Available at: <https://dataaspirant.com/svm-kernels/#t-1608054630734> [Accessed 17 December 2020].

Burnwal, S., 2020. Speech Emotion Recognition. [online] Kaggle.com. Available at: <<https://www.kaggle.com/code/shivamburnwal/speech-emotion-recognition/notebook>> [Accessed 1 January 2020].

Chavhan, Y., Dhore, M. and Yesaware, P., 2010. Speech Emotion Recognition using Support Vector Machine. *International Journal of Computer Applications*, 1(20), pp.8-11.

Chen, S. and Jin, Q. (2015). *Multi-modal Dimensional Emotion Recognition using Recurrent Neural Networks*. Brisbane, Australia.

Cowie, R., Douglas-Cowie, E., Tsapatsoulis, N., Votsis, G., Kollias, S., Fellenz, W. and Taylor, J., 2001. Emotion recognition in human-computer interaction. *IEEE Signal Processing Magazine*, 18(1), pp.32-80.

Dertat, A., 2017. Applied Deep Learning - Part 4: Convolutional Neural Networks. [online] Applied Deep Learning - Part 4: Convolutional Neural Networks. Available at: <<https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2>> [Accessed 8 November 2017].

Fayek, H., Lech, M. and Cavedon, L., 2017. Evaluating deep learning architectures for Speech Emotion Recognition. *Neural Networks*, 92, pp.60-68.

Femia, K., 2020. Study Proves That Horses Can Understand Human Emotions. [Online] Available at: <https://ihearthorses.com/horses-understand-human-emotions/> [Accessed 25 November 2020].

G.S., D. S., P., C., and B., V. (2016). SVM Based Speech Emotion Recognition Compared with GMM-UBM and NN. *IJESC*, 6.

Hochreiter, S., Younger, A.S. and Conwell, P.R., 2001, August. Learning to learn using gradient descent. In *International conference on artificial neural networks* (pp. 87-94). Springer, Berlin, Heidelberg.

Hu, H., Xu, M. and Wu, W., 2007. GMM Supervector Based SVM with Spectral Features for Speech Emotion Recognition. *2007 IEEE International Conference on Acoustics, Speech and Signal Processing - ICASSP '07*,.

Issa, D., Fatih Demirci, M. and Yazici, A., 2020. Speech emotion recognition with deep convolutional neural networks. *Biomedical Signal Processing and Control*, 59, p.101894.

Kerkeni, L., Serrestou, Y., Mbarki, M., Raoof, K. and Mahjoub, M., 2018. Speech Emotion Recognition: Methods and Cases Study. *Proceedings of the 10th International Conference on Agents and Artificial Intelligence*,.[176]

Kiran, U., 2021. *MFCC Technique for Speech Recognition*. [Online] Available at: <https://www.analyticsvidhya.com/blog/2021/06/mfcc-technique-for-speech-recognition/> [Accessed 13 June 2021].

Lugovic, S., Dunder, I. and Horvat, M., 2016. Techniques and applications of emotion recognition in speech. *2016 39th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*,.

Mao, Q., Dong, M., Huang, Z. and Zhan, Y., 2014. Learning Salient Features for Speech Emotion & Recognition Using Convolutional & Neural Networks. IEEE Transactions on Multimedia, 16(8), pp.2203-2213.

Marius, H., 2020. Multiclass Classification with Support Vector Machines (SVM), Kernel Trick & Kernel Functions. [online], Dual Problem and Kernel Functions. Available at: <<https://towardsdatascience.com/multiclass-classification-with-support-vector-machines-svm-kernel-trick-kernel-functions-f9d5377d6f02>> [Accessed 9 June 2020].

Mesevage, T. G., 2021. What Is Data Preprocessing & What Are The Steps Involved? [Online] Available at: <https://monkeylearn.com/blog/data-preprocessing/> [Accessed 24 May 2021].

MevonAI, <https://tinyurl.com/ze94b7ud>

Mirsamadi, S., Barsoum, E. and Zhang, C., 2017. Automatic speech emotion recognition using recurrent neural networks with local attention. 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP),.

Mustaqeem and Kwon, S., 2019. A CNN-Assisted Enhanced Audio Signal Processing for Speech Emotion Recognition. Sensors, 20(1), p.183.

Nakatsu, R., Nicholson, J. and Tosa, N., 1999. Emotion recognition and its application to computer agents with spontaneous interactive capabilities. Proceedings of the seventh ACM international conference on Multimedia (Part 1) - MULTIMEDIA '99,.

Nwe, T., Foo, S. and De Silva, L., 2003. Speech emotion recognition using hidden Markov models. Speech Communication, 41(4), pp.603-623.

Paperswithcode.com. n.d. Papers with Code - Speech Emotion Recognition. [online] Available at: <<https://paperswithcode.com/task/speech-emotion-recognition>>.

RESEARCH, M., 2022. TOP 10 COMPANIES IN SPEECH AND VOICE RECOGNITION MARKET. [Online] Available at: <https://meticulousblog.org/top-10-companies-in-speech-and-voice-recognition-market/> [Accessed 10 February 2022].

Rocha, W., Prado, C. and Blonder, N., 2020. Comparison of Chemometric Problems in Food Analysis using Non-Linear Methods. *Molecules*, 25(13), p.3025.

Sai, R. N. D., Shahbaaz, S. & Prakash, U., 2022. Speech Emotion Recognition using LSTM and RNN. *Compliance Engineering Journal*, 13(2), pp. 37-42.

Sepp, H. and Jurgen, S. (1997). Long Short-term Memory Neural Computation

Shen, P., Changjun, Z. and Chen, X., 2011. Automatic Speech Emotion Recognition using Support Vector Machine. *Proceedings of 2011 International Conference on Electronic & Mechanical Engineering and Information Technology*,.

Srivastava, P., 2017. Essentials of Deep Learning : Introduction to Long Short Term Memory. [Online] Available at: <https://www.analyticsvidhya.com/blog/2017/12/fundamentals-of-deep-learning-introduction-to- lstm/> [Accessed 18 May 2020].

Sudhakar, R. and Anil, M., 2015. Analysis of Speech Features for Emotion Detection: A Review. *2015 International Conference on Computing Communication Control and Automation*,.

Suresh, A., 2020. What is a confusion matrix?. [Online] Available at: <https://medium.com/analytics-vidhya/what-is-a-confusion-matrix-d1c0f8feda5> [Accessed 17 November 2020].

Swamy, S., 2022. RECENT TRENDS IN TIME SERIES FORECASTING-A SURVEY. [online] Available at: <https://www.researchgate.net/figure/Rolled-out-RNN-at-different-time-steps-Long-Short-Term-Memory-8-is-one-of-the-popular_fig1_341151347> [Accessed 1 April 2020].

Takimoglu, A., 2021. *What is Data Augmentation? Techniques & Examples in 2022*. [Online] Available at: <https://research.aimultiple.com/data-augmentation/#:~:text=Data%20augmentation%20is%20useful%20to%20improve%20performance%20and,sufficient%2C%20the%20model%20performs%20better%20and%20more%20accurately.> [Accessed 20 june 2022].

Trigeorgis, G., Ringeval, F., Brueckner, R., Marchi, E., Nicolaou, M., Schuller, B. and Zafeiriou, S., 2016. Adieu features? End-to-end speech emotion recognition using a deep convolutional recurrent network. 2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP),.

Wei Han, Cheong-Fat Chan, Chiu-Sing Choy and Kong-Pang Pun, n.d. An efficient MFCC extraction method in speech recognition. 2006 IEEE International Symposium on Circuits and Systems,.

Williams, C.E. and Stevens, K.N., 1981. Vocal correlates of emotional states. Speech evaluation in psychiatry, pp.221-240.

8. Appendix

Below is the attached file for code.



Emotion_Recognition
_in_Speech_20047638_

Below is the code for this project:

Data Science Project

Recognition of Emotions in Speech

Student ID: 20047638

The objective of this project is to comparative research on the topic of detecting emotion in speeches using the model CNN, RNN, MLP and SVM comparing their accuracy, predictive quality and training-testing speed when trained on the same dataset.

Importing and Loading Data

In []:

The datasets CREMA-D and TESS are taken from kaggle. Here, the datasets are first imported to the google colab and then converted and merged into one dataframe

```
# Importing all the necessary libraries
```

```
# Importing Numpy, Pandas, Tensorflow, Keras Libraries
```

```
import numpy as np
```

```
import pandas as pd
```

```
import seaborn as sn
```

```
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
```

```
from tensorflow import keras
```

```
from keras.callbacks import ReduceLROnPlateau
```

```
from tensorflow.keras import layers, models
```

```
from keras.models import Sequential
```

```
from keras.layers import Dense, Conv1D, MaxPooling1D, Flatten, Dropout,
BatchNormalization, Activation, LSTM
from keras.callbacks import ModelCheckpoint
import librosa
import librosa.display
from matplotlib.pyplot import specgram
import os
import sys
import warnings
import keras
# Importing sklearn library
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
import time
# ignore warnings
if not sys.warnoptions:
    warnings.simplefilter("ignore")
warnings.filterwarnings("ignore", category=DeprecationWarning)
```

In []:

We first start with the CREMA-D dataset. The dataset is downloaded and then converted to a dataframe.

Looking in indexes: <https://pypi.org/simple>, (<https://pypi.org/simple>,) <https://us-python.pkg.dev/colab-wheels/public/s>

imple/ (<https://us-python.pkg.dev/colab-wheels/public/simple/>)

Requirement already satisfied: kaggle in /usr/local/lib/python3.7/dist-packages (1.5.12)

Requirement already satisfied: urllib3 in /usr/local/lib/python3.7/dist-packages (from kaggle) (1.24.3)

Requirement already satisfied: python-dateutil in /usr/local/lib/python3.7/dist-packages (from kaggle) (2.8.2)

Requirement already satisfied: certifi in /usr/local/lib/python3.7/dist-packages (from kaggle) (2022.6.15)

Requirement already satisfied: python-slugify in /usr/local/lib/python3.7/dist-packages (from kaggle) (6.1.2)

Requirement already satisfied: requests in /usr/local/lib/python3.7/dist-packages (from kaggle) (2.23.0)

Requirement already satisfied: tqdm in /usr/local/lib/python3.7/dist-packages (from kaggle) (4.64.0)

Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.7/dist-packages (from kaggle) (1.15.0)

Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.7/dist-packages (from python-slugify->kaggle) (1.3)

Requirement already satisfied: chardet<4,>=3.0.2 in /usr/local/lib/python3.7/dist-packages (from requests->kaggle) (3.0.4)

Requirement already satisfied: idna<3,>=2.5 in /usr/local/lib/python3.7/dist-packages (from requests->kaggle) (2.10)

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle.json to kaggle.json

The uploaded file name: "kaggle.json" with length 68 bytes

```
# Install Kaggle API to be able to access Kaggle datasets
!pip install kaggle
from google.colab import files
files_upload = files.upload()
for fn in files_upload.keys():
    print('The uploaded file name: "{name}" with length {length} bytes'.format(
        name=fn, length=len(files_upload[fn])))
```

```
# Then move kaggle.json into the folder where the API expects to find it.
```

```
!mkdir -p ~/.kaggle/ && mv kaggle.json ~/.kaggle/ && chmod 600
```

```
~/.kaggle/kaggle.json
```

```
In [ ]:
```

```
inflating: AudioWAV/1031_ITH_FEA_XX.wav
```

```
inflating: AudioWAV/1031_ITH_HAP_XX.wav
```

```
inflating: AudioWAV/1031_ITH_NEU_XX.wav
```

```
inflating: AudioWAV/1031_ITH_SAD_XX.wav
```

```
inflating: AudioWAV/1031_ITS_ANG_XX.wav
```

```
inflating: AudioWAV/1031_ITS_DIS_XX.wav
```

```
inflating: AudioWAV/1031_ITS_FEA_XX.wav
```

```
inflating: AudioWAV/1031_ITS_HAP_XX.wav
```

```
inflating: AudioWAV/1031_ITS_NEU_XX.wav
```

```
inflating: AudioWAV/1031_ITS_SAD_XX.wav
```

```
inflating: AudioWAV/1031_IWL_ANG_XX.wav
```

```
inflating: AudioWAV/1031_IWL_DIS_XX.wav
```

```
inflating: AudioWAV/1031_IWL_FEA_XX.wav
```

```
inflating: AudioWAV/1031_IWL_HAP_XX.wav
```

```
inflating: AudioWAV/1031_IWL_NEU_XX.wav
```

```
inflating: AudioWAV/1031_IWL_SAD_XX.wav
```

```
inflating: AudioWAV/1031_IWW_ANG_XX.wav
```

```
inflating: AudioWAV/1031_IWW_DIS_XX.wav
```

```
inflating: AudioWAV/1031_IWW_FEA_XX.wav
```

```
# Downloading CREMA-D dataset and unzipping to make it available in
```

```
# the google colab directory
```

```
!kaggle datasets download "ejlok1/cremad"
```

```
!unzip cremad.zip
```

```
In [ ]:
```

```
Out[27]: Emotions Path
```

```
0 fear /content/AudioWAV/1054_TIE_FEA_XX.wav
```

```
1 disgust /content/AudioWAV/1079_ITH_DIS_XX.wav
```

```
# Creating dataframe with the given data
```

```
crema = '/content/AudioWAV'
```

```

crema_directory_list = os.listdir(crema)
file_emotion = []
file_path = []
for file in crema_directory_list:
    # storing file paths
    file_path.append(crema + '/' + file)
    # storing file emotions
    part=file.split('_')
    if part[2] == 'SAD':
        file_emotion.append('sad')
    elif part[2] == 'ANG':
        file_emotion.append('angry')
    elif part[2] == 'DIS':
        file_emotion.append('disgust')
    elif part[2] == 'FEA':
        file_emotion.append('fear')
    elif part[2] == 'HAP':
        file_emotion.append('happy')
    elif part[2] == 'NEU':
        file_emotion.append('neutral')
    else:
        file_emotion.append('Unknown')

# dataframe for emotion of files
emotion_df = pd.DataFrame(file_emotion, columns=['Emotions'])
# dataframe for path of files.
path_df = pd.DataFrame(file_path, columns=['Path'])
Crema_df = pd.concat([emotion_df, path_df], axis=1)
Crema_df.head()
Now, we download the TESS dataset and create a dataframe
In [ ]:
Emotions Path
2 disgust /content/AudioWAV/1040_IEO_DIS_LO.wav

```

3 fear /content/AudioWAV/1087_WSI_FEA_XX.wav

4 angry /content/AudioWAV/1033_DFA_ANG_XX.wav

Streaming output truncated to the last 5000 lines.

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_back_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_bar_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_base_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_bath_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_bean_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_beg_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_bite_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_boat_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_bone_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_book_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_bought_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_burn_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_cab_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_calm_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_came_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_cause_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_chain_angry.wav

inflating: TESS Toronto emotional speech set data/OAF_angry/OAF_chair_angry.wav

Downloading TESS dataset and unzipping to make it available in

the google colab directory

!kaggle datasets download "ejlok1/toronto-emotional-speech-set-tess"

!unzip toronto-emotional-speech-set-tess.zip

In []:

Merging data into one dataframe

Out[6]: Emotions Path

0 angry /content/TESS Toronto emotional speech set dat...

1 angry /content/TESS Toronto emotional speech set dat...

2 angry /content/TESS Toronto emotional speech set dat...

```

3 angry /content/TESS Toronto emotional speech set dat...
4 angry /content/TESS Toronto emotional speech set dat...
# Creating dataframe with the given data
Tess = '/content/TESS Toronto emotional speech set data'
tess_directory_list = os.listdir(Tess)
file_emotion = []
file_path = []
for dir in tess_directory_list:
    directories = os.listdir(Tess + '/' + dir) #Tess +
    for file in directories:
        part = file.split('.')[0]
        part = part.split('_')[2]
        if part=='ps':
            file_emotion.append('surprise')
        else:
            file_emotion.append(part)
        file_path.append(Tess + '/' + dir + '/' + file)

# dataframe for emotion of files
emotion_df = pd.DataFrame(file_emotion, columns=['Emotions'])
# dataframe for path of files.
path_df = pd.DataFrame(file_path, columns=['Path'])
Tess_df = pd.concat([emotion_df, path_df], axis=1)
Tess_df.head()

```

```

In [ ]: # creating a Dataframe using the 2 dataframes created.
audiopath_data = pd.concat([Crema_df, Tess_df], axis = 0)
audiopath_data.to_csv("data_path.csv",index=False)
audiopath_data.head()

```

```

Out[7]: Emotions Path
0 happy /content/AudioWAV/1054_IWW_HAP_XX.wav
1 sad /content/AudioWAV/1056_DFA_SAD_XX.wav

```

```
2 sad /content/AudioWAV/1020_IEO_SAD_LO.wav
3 fear /content/AudioWAV/1088_TAI_FEA_XX.wav
4 neutral /content/AudioWAV/1003_WSI_NEU_XX.wav
# creating a Dataframe using the 2 dataframes created.
audiopath_data = pd.concat([Crema_df, Tess_df], axis = 0)
audiopath_data.to_csv("data_path.csv",index=False)
audiopath_data.head()
```

```
In [ ]: # creating a Dataframe using the 2 dataframes created.
audiopath_data = pd.concat([Crema_df, Tess_df], axis = 0)
audiopath_data.to_csv("data_path.csv",index=False)
audiopath_data.head()
```

```
Out[31]: Text(0, 0.5, 'Amplitude')
# Plotting an audio file waveplot to view the data - CREMA-D/Happy
file_name = '/content/AudioWAV/1054_IWW_HAP_XX.wav'
data, sampling_rate = librosa.load(file_name)
plt.figure(figsize=(15, 5))
librosa.display.waveplot(data, sr=sampling_rate)
plt.title('Happy')
plt.ylabel('Amplitude')
```

```
In [ ]: # Plotting an audio file waveplot to view the data - CREMA-D/Fear
```

```
file_name = '/content/AudioWAV/1088_TAI_FEA_XX.wav'
data, sampling_rate = librosa.load(file_name)
plt.figure(figsize=(15, 5))
librosa.display.waveplot(data, sr=sampling_rate)
plt.title('Fear')
plt.ylabel('Amplitude')
Out[32]: Text(0, 0.5, 'Amplitude')
```

Feature Extraction

Now, We extract the features, from the given audio files in the dataframes

In []: ## Code taken from Kaggle##

Definitions of functions to be used for feature extraction

def noise(data):

noise_amp = 0.035*np.random.uniform()*np.amax(data)

data = data + noise_amp*np.random.normal(size=data.shape[0])

return data

def stretch(data, rate=0.8):

return librosa.effects.time_stretch(data, rate)

def shift(data):

shift_range = int(np.random.uniform(low=-5, high = 5)*1000)

return np.roll(data, shift_range)

def pitch(data, sampling_rate, pitch_factor=0.7):

return librosa.effects.pitch_shift(data, sampling_rate, pitch_factor)

def higher_speed(data, speed_factor = 1.25):

return librosa.effects.time_stretch(data, speed_factor)

def lower_speed(data, speed_factor = 0.75):

return librosa.effects.time_stretch(data, speed_factor)

In []: ## Code taken from Kaggle##

Extracting Features using MFCCs

def extract_features(data):

result = np.array([])

mfccs = librosa.feature.mfcc(y=data, sr=22050, n_mfcc=58)

mfccs_processed = np.mean(mfccs.T,axis=0)

result = np.array(mfccs_processed)

return result

def get_features(path):

duration and offset are used to take care of the no audio in start and the ending of each audio files as seen above

data, sample_rate = librosa.load(path, duration=3, offset=0.5, res_type='kaiser_fast')

#without augmentation

```
res1 = extract_features(data)
result = np.array(res1)

#noised
noise_data = noise(data)
res2 = extract_features(noise_data)
result = np.vstack((result, res2)) # stacking vertically

#stretched
stretch_data = stretch(data)
res3 = extract_features(stretch_data)
result = np.vstack((result, res3))

#shifted
shift_data = shift(data)
res4 = extract_features(shift_data)
result = np.vstack((result, res4))

#pitched
pitch_data = pitch(data, sample_rate)
res5 = extract_features(pitch_data)
result = np.vstack((result, res5))

#speed up
higher_speed_data = higher_speed(data)
res6 = extract_features(higher_speed_data)
```

```
Out[11]: 0 1 2 3 4 5 6 7 8 9 ... 49 50
0 -372.859833 158.552551 -5.121318 38.753338 -12.569175 10.467646 -17.361149 -
3.500664 -10.303802 -4.179052 ... -1.381174 -2.042949 1.837
1 -235.423931 50.915942 23.368699 9.832177 3.868557 -3.130746 -7.366017 -
7.309678 -5.633727 -1.876884 ... -1.490963 -0.792193 1.351
```

```
2 -406.562378 157.783981 -6.387405 38.337200 -13.279373 8.636255 -18.171515 -
3.022032 -11.198946 -4.556575 ... -1.518174 -2.668355 2.245
3 -371.539337 157.163910 -4.820157 38.652512 -12.831516 10.374809 -17.519386 -
3.489297 -10.154447 -4.095973 ... -1.122553 -1.963099 1.756
4 -397.103271 153.161499 -4.517867 33.218170 -14.287848 3.471455 -20.819828 -
1.789339 -13.161255 0.570050 ... -0.895595 3.226112 1.632
```

```
5 rows × 59 columns
```

```
result = np.vstack((result, res6))
```

```
#speed down
```

```
lower_speed_data = higher_speed(data)
```

```
res7 = extract_features(lower_speed_data)
```

```
result = np.vstack((result, res7))
```

```
return result
```

```
## Code taken from Kaggle##
```

```
## Calling the above defined functions for feature extraction
```

```
X_features, Y_labels = [], []
```

```
for path, emotion in zip(audiopath_data.Path, audiopath_data.Emotions):
```

```
    feature = get_features(path)
```

```
    for elements in feature:
```

```
        X_features.append(elements)
```

```
    # appending emotion 3 times as we have made 3 augmentation techniques on each
    audio file.
```

```
    Y_labels.append(emotion)
```

```
## Creating a dataframe with features
```

```
Features_DF = pd.DataFrame(X_features)
```

```
Features_DF['labels'] = Y_labels
```

```
Features_DF.to_csv('features.csv', index=False)
```

```
Features_DF.head()
```

Data Preparation

In []:

```

# Rereading the saved features from the features csv
Features_DF = pd.read_csv('features.csv')
X_features = Features_DF.iloc[:, :-1].values
Y_labels = Features_DF['labels'].values
emotions = ['Angry', 'Disgust', 'Fear', 'Happy', 'Neutral', 'Sad', 'Surprise']
# As this is a multiclass classification, using onehotencoder for our labels.
label_encoder = OneHotEncoder()
Y_labels = label_encoder.fit_transform(np.array(Y_labels)
    .reshape(-1,1))\
    .toarray()
# splitting the data into train and test set
x_train, x_test, y_train, y_test = train_test_split(X_features,
    Y_labels,
    random_state=0,
    shuffle=True)
x_train.shape, y_train.shape, x_test.shape, y_test.shape
# scaling our data
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
x_train.shape, y_train.shape, x_test.shape, y_test.shape

Out[4]: ((53770, 58), (53770, 7), (17924, 58), (17924, 7))
Out[5]: ((53770, 58), (53770, 7), (17924, 58), (17924, 7))

```

Building Models for our dataset
CNN Model

Now, we build the CNN model and train it using the given train data. This model will then be validated against the test data from our dataset

In []:

```

Out[6]: ((53770, 58, 1), (53770, 7), (17924, 58, 1), (17924, 7))
# reshaping the data for CNN model compatibility

```

```

x_train_cnn = np.expand_dims(x_train, axis=2)
x_test_cnn = np.expand_dims(x_test, axis=2)
y_train_cnn = y_train
y_test_cnn = y_test
x_train_cnn.shape, y_train_cnn.shape, x_test_cnn.shape, y_test_cnn.shape
In [ ]:
Model: "sequential"

```

```

Layer (type) Output Shape Param #
=====

```

```

conv1d (Conv1D) (None, 58, 256) 1536

```

```

activation (Activation) (None, 58, 256) 0
conv1d_1 (Conv1D) (None, 58, 128) 163968
activation_1 (Activation) (None, 58, 128) 0
dropout (Dropout) (None, 58, 128) 0
max_pooling1d (MaxPooling1D) (None, 7, 128) 0
)

```

```

conv1d_2 (Conv1D) (None, 7, 128) 82048

```

```

# CNN Model

```

```

from tensorflow import keras
CNN_model = Sequential()
CNN_model.add(Conv1D(256, 5,padding='same',
input_shape=(x_train.shape[1],1)))
CNN_model.add(Activation('relu'))
CNN_model.add(Conv1D(128, 5,padding='same'))
CNN_model.add(Activation('relu'))
CNN_model.add(Dropout(0.1))
CNN_model.add(MaxPooling1D(pool_size=(8)))
CNN_model.add(Conv1D(128, 5,padding='same',))
CNN_model.add(Activation('relu'))
CNN_model.add(Conv1D(128, 5,padding='same',))

```

```
CNN_model.add(Activation('relu'))
CNN_model.add(Flatten())
CNN_model.add(Dense(7))
CNN_model.add(Activation('softmax'))
opt = keras.optimizers.RMSprop(learning_rate=0.01, decay=1e-6)
CNN_model.compile(loss='categorical_crossentropy',
optimizer=opt,metrics=['accuracy'])
CNN_model.summary()
activation_2 (Activation) (None, 7, 128) 0
```

```
conv1d_3 (Conv1D) (None, 7, 128) 82048
activation_3 (Activation) (None, 7, 128) 0
flatten (Flatten) (None, 896) 0
dense (Dense) (None, 7) 6279
activation_4 (Activation) (None, 7) 0
```

```
=====
Total params: 335,879
Trainable params: 335,879
Non-trainable params: 0
```

```
In [ ]:
_accuracy: 0.8395 - lr: 0.0016
Epoch 45/50
211/211 [=====] - 97s 462ms/step - loss: 0.2109 -
accuracy: 0.9273 - val_loss: 0.5854 - val
_accuracy: 0.8331 - lr: 0.0016
Epoch 46/50
211/211 [=====] - 98s 465ms/step - loss: 0.2088 -
accuracy: 0.9299 - val_loss: 0.5896 - val
_accuracy: 0.8375 - lr: 0.0016
Epoch 47/50
211/211 [=====] - 98s 464ms/step - loss: 0.2030 -
accuracy: 0.9292 - val_loss: 0.5642 - val
```

```

_accuracy: 0.8427 - lr: 0.0016
Epoch 48/50
211/211 [=====] - 98s 464ms/step - loss: 0.1985 -
accuracy: 0.9312 - val_loss: 0.6093 - val
_accuracy: 0.8402 - lr: 0.0016
Epoch 49/50
211/211 [=====] - 98s 464ms/step - loss: 0.1964 -
accuracy: 0.9320 - val_loss: 0.6045 - val
_accuracy: 0.8374 - lr: 0.0016
Epoch 50/50
211/211 [=====] - 98s 464ms/step - loss: 0.1957 -
accuracy: 0.9325 - val_loss: 0.6062 - val
_accuracy: 0.8447 - lr: 0.0016
CNN model trained in 4888.570144891739 s
# Training the CNN model on our training data
learning_rate = ReduceLROnPlateau(monitor='loss',
factor=0.4,
verbose=0,
patience=2,
min_lr=0.0000001)
starting_time = time.time()
history_CNN = CNN_model.fit(x_train_cnn, y_train_cnn,
batch_size=256,
epochs=50,
validation_data=(x_test_cnn, y_test_cnn),
callbacks=[learning_rate])
print("CNN model trained in %s s " % str(time.time() - starting_time))
In [ ]: # Plotting the graphs for training and test accuracy
epochs = [i for i in range(50)]
fig , ax = plt.subplots(1,2)
train_acc = history_CNN.history['accuracy']
train_loss = history_CNN.history['loss']
test_acc = history_CNN.history['val_accuracy']

```

```

test_loss = history_CNN.history['val_loss']
fig.set_size_inches(20,6)
ax[0].plot(epochs , train_loss , label = 'Training Loss')
ax[0].plot(epochs , test_loss , label = 'Testing Loss')
ax[0].set_title('Training & Testing Loss')
ax[0].legend()
ax[0].set_xlabel("Epochs")
ax[1].plot(epochs , train_acc , label = 'Training Accuracy')
ax[1].plot(epochs , test_acc , label = 'Testing Accuracy')
ax[1].set_title('Training & Testing Accuracy')
ax[1].legend()
ax[1].set_xlabel("Epochs")
plt.show()
In [ ]:
In [ ]:
CNN model tested in 7.4804136753082275 s
The actual emotions: [['angry']
['fear']
['angry']
...
['neutral']
['disgust']
['neutral']] The predicted emotions: [['angry']
['fear']
['angry']
...
['neutral']
['disgust']
['neutral']]
Accuracy: 84.47%
# Determining the predictive quality of the network
starting_time = time.time()
y_pred_cnn = CNN_model.predict(x_test_cnn)

```

```

print("CNN model tested in %s s " % str(time.time() - starting_time))
y_pred_cnn = label_encoder.inverse_transform(y_pred_cnn)
ytest_dec_cnn = label_encoder.inverse_transform(y_test_cnn)
print('The actual emotions:', ytest_dec_cnn, 'The predicted emotions:', y_pred_cnn)
# Determining the accuracy of our model
accuracy_CNN = accuracy_score(y_true = ytest_dec_cnn ,
    y_pred = y_pred_cnn)
# Printing the accuracy of our MLP model
print("Accuracy: {:.2f}%".format(accuracy_CNN*100))
MLP (Multi-Layer Perceptron) Model
In [ ]:
In [ ]:
In [ ]:
MLP model trained in 393.80851912498474 s
# Initialising the train and test data
x_train_mlp = x_train
x_test_mlp = x_test
y_train_mlp = y_train
y_test_mlp = y_test
# Defining the Multi Layer Perceptron Classifier
MLP_model = MLPClassifier(alpha=0.06,
    batch_size=256,
    epsilon=1e-08,
    hidden_layer_sizes=(250, 150),
    learning_rate='adaptive',
    max_iter=400)
# Training the MLP model against the training data
import time
starting_time = time.time()
MLP_model.fit(x_train_mlp,y_train_mlp)
print("MLP model trained in %s s " % str(time.time() - starting_time))
In [ ]:
MLP model tested in 0.19156312942504883 s

```

```

[[1 0 0 ... 0 0 0]
[0 0 1 ... 0 0 0]
[1 0 0 ... 0 0 0]
...
[0 0 0 ... 1 0 0]
[0 1 0 ... 0 0 0]
[0 0 0 ... 1 0 0]]
The actual emotions: [['angry']
['fear']
['angry']
...
['neutral']
['disgust']
['neutral']]
The predicted emotions: [['angry']
['fear']
['angry']
...
# Testing the predictive quality of the model:
starting_time = time.time()
y_pred_mlp = MLP_model.predict(x_test_mlp)
print("MLP model tested in %s s " % str(time.time() - starting_time))
df_temp = pd.DataFrame(y_pred_mlp)
for row in range(len(df_temp)):
    if np.sum(df_temp.loc[row,:]) == 0:
        df_temp.loc[row,2] = 1
y_pred_mlp = pd.DataFrame.to_numpy(df_temp)
print(y_pred_mlp)
y_pred_mlp = label_encoder.inverse_transform(y_pred_mlp)
y_test_mlp = label_encoder.inverse_transform(y_test_mlp)
print('The actual emotions:', y_test_mlp)
print('The predicted emotions:', y_pred_mlp)
In [ ]:

```

SVM (Support Vector Machine) Model

In []:

In []:

In []:

In []:

['neutral']

['disgust']

['neutral']]

Accuracy: 86.29%

Out[51]: ((57355, 58), (57355,), (14339, 58), (14339,))

Determining the accuracy of our model

```
accuracy_MLP= accuracy_score(y_true = y_test_mlp,  
y_pred = y_pred_mlp)
```

Printing the accuracy of our MLP model

```
print("Accuracy: {:.2f}%".format(accuracy_MLP*100))
```

Rereading the saved features from the features csv

```
Features_DF = pd.read_csv('features.csv')
```

```
X_features = Features_DF.iloc[:, :-1].values
```

```
Y_labels = Features_DF['labels'].values
```

Initialising the train and test data

```
x_train_svm, x_test_svm, y_train_svm, y_test_svm = train_test_split(X_features,  
Y_labels,
```

```
test_size = 0.20)
```

scaling our data

```
scaler = StandardScaler()
```

```
x_train_svm = scaler.fit_transform(x_train_svm)
```

```
x_test_svm = scaler.transform(x_test_svm)
```

```
x_train_svm.shape, y_train_svm.shape, x_test_svm.shape, y_test_svm.shape
```

```
from sklearn.svm import SVC
```

```
svmclassifier = SVC(kernel = 'rbf')
```

LSTM (Long Short Term Memory) Model

In []:

Trained model in 369.50613021850586 s

```

Tested model in 73.30016803741455 s
Predicted values: ['sad' 'fear' 'disgust' 'fear' 'fear' 'disgust' 'fear' 'angry' 'disgust'
'sad']
Actual values: ['sad' 'sad' 'disgust' 'fear' 'fear' 'disgust' 'fear' 'disgust' 'disgust'
'sad']
----accuracy score 72.80842457633028 ----
import time
starting_time = time.time()
svmclassifier.fit(x_train_svm, y_train_svm)
print("Trained model in %s s " % str(time.time() - starting_time))
# Predictive model
starting_time = time.time()
y_pred_svm = svmclassifier.predict(x_test_svm)
print("Tested model in %s s " % str(time.time() - starting_time))
print('Predicted values: ',y_pred_svm[:10])
print('Actual values: ', y_test_svm[:10])
acc = float(accuracy_score(y_test_svm,y_pred_svm))*100
print("----accuracy score %s ----" % acc)
# Rereading the saved features from the features csv
Features_DF = pd.read_csv('features.csv')
X_features = Features_DF.iloc[:, :-1].values
Y_labels = Features_DF['labels'].values
In [ ]:
In [ ]:
Out[97]: ((51619, 58, 1), (14339, 58, 1), (51619, 7), (14339, 7))
# As this is a multiclass classification, using onehotencoder for our labels.
label_encoder = OneHotEncoder()
Y_labels = label_encoder.fit_transform(np.array(Y_labels)
.reshape(-1,1))\
.toarray()
x_train_rnn, x_test_rnn, y_train_rnn, y_test_rnn = train_test_split(
X_features,
Y_labels,

```

```

test_size=0.2)
x_train_rnn, x_validation_rnn, y_train_rnn, y_validation_rnn = train_test_split(
    x_train_rnn,
    y_train_rnn,
    test_size=0.1)
# making our data compatible to model.
x_train_rnn = np.expand_dims(x_train_rnn, axis=-1)
x_test_rnn = np.expand_dims(x_test_rnn, axis=-1)
x_validation_rnn = np.expand_dims(x_validation_rnn, axis=-1)
x_train_rnn.shape, x_test_rnn.shape, y_train_rnn.shape, y_test_rnn.shape
In [ ]:
In [ ]:
Model: "sequential_5"

```

Layer (type)	Output Shape	Param #
--------------	--------------	---------

lstm_5 (LSTM)	(None, 200)	161600
---------------	-------------	--------

dropout_5 (Dropout)	(None, 200)	0
---------------------	-------------	---

dense_15 (Dense)	(None, 512)	102912
------------------	-------------	--------

dense_16 (Dense)	(None, 512)	262656
------------------	-------------	--------

dense_17 (Dense)	(None, 7)	3591
------------------	-----------	------

Total params: 530,759

Trainable params: 530,759

Non-trainable params: 0

```
# Building the RNN model
```

```
model_RNN = Sequential()
```

```
model_RNN.add(LSTM(200, input_shape=(x_train_rnn.shape[1],
x_train_rnn.shape[2],),))
```

```
model_RNN.add(Dropout(0.5))
```

```
model_RNN.add(Dense(512))
```

```

model_RNN.add(Dense(512))
model_RNN.add(Dense(y_train_rnn.shape[1], activation='softmax'))
model_RNN.summary()
model_RNN.compile(loss='binary_crossentropy', optimizer='Nadam',
metrics=['accuracy'])
# Defining callbacks - learning rate
learning_rate_rnn = ReduceLROnPlateau(monitor='loss',
factor=0.4,
verbose=0,
patience=2,
min_lr=0.000001)
In [ ]:
_accuracy: 0.8878 lr: 0.0010
Epoch 55/60
454/454 [=====] - 64s 141ms/step - loss: 0.0175 -
accuracy: 0.9796 - val_loss: 0.1016 - val
_accuracy: 0.9070 - lr: 4.0000e-04
Epoch 56/60
454/454 [=====] - 65s 143ms/step - loss: 0.0114 -
accuracy: 0.9873 - val_loss: 0.1102 - val
_accuracy: 0.9070 - lr: 4.0000e-04
Epoch 57/60
454/454 [=====] - 64s 141ms/step - loss: 0.0117 -
accuracy: 0.9874 - val_loss: 0.1154 - val
_accuracy: 0.9033 - lr: 4.0000e-04
Epoch 58/60
454/454 [=====] - 64s 141ms/step - loss: 0.0130 -
accuracy: 0.9859 - val_loss: 0.1111 - val
_accuracy: 0.9093 - lr: 4.0000e-04
Epoch 59/60
454/454 [=====] - 64s 141ms/step - loss: 0.0066 -
accuracy: 0.9932 - val_loss: 0.1234 - val
_accuracy: 0.9118 - lr: 1.6000e-04

```

```

Epoch 60/60
454/454 [=====] - 66s 145ms/step - loss: 0.0046 -
accuracy: 0.9954 - val_loss: 0.1325 - val
_accuracy: 0.9135 - lr: 1.6000e-04
RNN-LSTM model trained in 3803.1767585277557 s
starting_time = time.time()
history_rnn = model_RNN.fit(x_train_rnn, y_train_rnn,
validation_data=(x_validation_rnn, y_validation_rnn),
batch_size= 128,
epochs=60,
callbacks = [learning_rate_rnn])
print("RNN-LSTM model trained in %s s " % str(time.time() - starting_time))
In [ ]: # Plotting the graphs for training and test accuracy
epochs = [i for i in range(60)]
fig , ax = plt.subplots(1,2)
train_acc_rnn = history_rnn.history['accuracy']
train_loss_rnn = history_rnn.history['loss']
test_acc_rnn = history_rnn.history['val_accuracy']
test_loss_rnn = history_rnn.history['val_loss']
fig.set_size_inches(20,6)
ax[0].plot(epochs , train_loss_rnn , label = 'Training Loss')
ax[0].plot(epochs , test_loss_rnn , label = 'Testing Loss')
ax[0].set_title('Training & Testing Loss')
ax[0].legend()
ax[0].set_xlabel("Epochs")
ax[1].plot(epochs , train_acc_rnn , label = 'Training Accuracy')
ax[1].plot(epochs , test_acc_rnn , label = 'Testing Accuracy')
ax[1].set_title('Training & Testing Accuracy')
ax[1].legend()
ax[1].set_xlabel("Epochs")
plt.show()
In [ ]:
Test Accuracy: 98.13560843467712

```

```
test_loss, test_acc = model_RNN.evaluate(x_test_rnn, y_test_rnn, verbose=0)
print("Test Accuracy: ",test_acc*100)
In [ ]:
In [ ]:
RNN-LSTM model tested in 2.2305257320404053 s
The actual emotions: [['neutral']
['happy']
['neutral']
['happy']
['happy']
['angry']
['angry']
['disgust']
['sad']
['fear']]
The predicted emotions: [['neutral']
['happy']
['neutral']
['disgust']
['happy']
['angry']
['angry']
['disgust']
['sad']
['fear']]
Accuracy: 98.14%
starting_time = time.time()
y_pred_rnn = model_RNN.predict(x_test_rnn)
print("RNN-LSTM model tested in %s s " % str(time.time() - starting_time))
y_pred_rnn = label_encoder.inverse_transform(y_pred_rnn)
y_test_rnn = label_encoder.inverse_transform(y_test_rnn)
print('The actual emotions:', y_test_rnn[:10])
print('The predicted emotions:', y_pred_rnn[:10])
```

```

# Determining the accuracy of our model
accuracy_RNN = accuracy_score(y_true = y_test_rnn,
                               y_pred = y_pred_rnn)
# Printing the accuracy of our RNN model
print("Accuracy: {:.2f}%".format(accuracy_RNN*100))
Accuracy of the Given models and their comparision
Analysing Model Performances
In [ ]:
# CNN Model
## Classification report
print(classification_report(ytest_dec_cnn,y_pred_cnn))
class_cnn = classification_report(ytest_dec_cnn,y_pred_cnn, output_dict=True)
class_cnn = pd.DataFrame(class_cnn).transpose()
class_cnn.to_csv('Class_cnn.csv', index = True)
acc = float(accuracy_score(ytest_dec_cnn,y_pred_cnn))*100
print("----accuracy score %s ----" % acc)
## Confusion matrix and its heatmap
cm_cnn = confusion_matrix(ytest_dec_cnn,y_pred_cnn)
df_cm_cnn = pd.DataFrame(cm_cnn)
df_cm_cnn.to_csv('CNN.csv', index = False)
sn.heatmap(df_cm_cnn/np.sum(df_cm_cnn),
           annot= True, fmt='.2%',
           xticklabels= emotions,
           yticklabels=emotions)
plt.show()

In [ ]:
# MLP Model
## Classification report
print(classification_report(y_test_mlp,y_pred_mlp))
class_mlp = classification_report(y_test_mlp,y_pred_mlp, output_dict=True)
class_mlp = pd.DataFrame(class_mlp).transpose()
class_mlp.to_csv('Class_mlp.csv', index = False)

```

```
acc = float(accuracy_score(y_test_mlp,y_pred_mlp))*100
print("----accuracy score %s ----" % acc)
## Confusion matrix and its heatmap
cm_mlp = confusion_matrix(y_test_mlp,y_pred_mlp)
df_cm_mlp = pd.DataFrame(cm_mlp)
df_cm_mlp.to_csv('MLP.csv', index=False)
sn.heatmap(df_cm_mlp/np.sum(df_cm_mlp),
annot= True, fmt='.2%',
xticklabels= emotions,
yticklabels=emotions)
plt.show()
```

In []:

```
# SVM Model
## Classification report
print(classification_report(y_test_svm,y_pred_svm))
class_svm = classification_report(y_test_svm,y_pred_svm, output_dict=True)
class_svm = pd.DataFrame(class_svm).transpose()
class_svm.to_csv('Class_svm.csv', index = False)
acc = float(accuracy_score(y_test_svm,y_pred_svm))*100
print("----accuracy score %s ----" % acc)
## Confusion matrix and its heatmap
cm_svm = confusion_matrix(y_test_svm,y_pred_svm)
df_cm_svm = pd.DataFrame(cm_svm)
df_cm_svm.to_csv('SVM.csv', index=False)
sn.heatmap(df_cm_svm/np.sum(df_cm_svm),
annot= True, fmt='.2%',
xticklabels= emotions,
yticklabels=emotions)
plt.show()
```

In []:

```
#RNN Model
```

```

## Classification report
emotions = ['Angry', 'Disgust', 'Fear', 'Happy', 'Neutral', 'Sad', 'Surprise']
print(classification_report(y_test_rnn,y_pred_rnn))
class_rnn = classification_report(y_test_rnn,y_pred_rnn, output_dict=True)
class_rnn = pd.DataFrame(class_rnn).transpose()
class_rnn.to_csv('Class_rnn.csv', index = False)
acc = float(accuracy_score(y_test_rnn,y_pred_rnn))*100
print("----accuracy score %s ----" % acc)
## Confusion matrix and its heatmap
cm_rnn = confusion_matrix(y_test_rnn,y_pred_rnn)
df_cm_rnn = pd.DataFrame(cm_rnn)
df_cm_rnn.to_csv('RNN.csv', index=False)
sn.heatmap(df_cm_rnn/np.sum(df_cm_rnn),
annot= True, fmt='.2%',
xticklabels= emotions,
yticklabels=emotions)
plt.show()
Analysing stats of each emotion
In [ ]: emotions = ['Angry', 'Disgust', 'Fear', 'Happy', 'Neutral', 'Sad', 'Surprise']
class_cnn = pd.read_csv('Class_cnn.csv')
class_mlp = pd.read_csv('Class_mlp.csv')
class_svm = pd.read_csv('Class_svm.csv')
class_rnn = pd.read_csv('Class_rnn.csv')

precision = pd.DataFrame()
precision['CNN'] = class_cnn['precision']
precision['MLP'] = class_mlp['precision']
precision['SVM'] = class_svm['precision']
precision['RNN'] = class_rnn['precision']
precision = precision.iloc[0:7,:]
precision.index = emotions

recall = pd.DataFrame()

```

```
recall['CNN'] = class_cnn['recall']
recall['MLP'] = class_mlp['recall']
recall['SVM'] = class_svm['recall']
recall['RNN'] = class_rnn['recall']
recall = recall.iloc[0:7,:]
recall.index = emotions
```

```
score = pd.DataFrame()
score['CNN'] = class_cnn['f1-score']
score['MLP'] = class_mlp['f1-score']
score['SVM'] = class_svm['f1-score']
score['RNN'] = class_rnn['f1-score']
score = score.iloc[0:7,:]
score.index = emotions
```

```
In [ ]:
```

```
# Plotting precision of each emotions in each models
```

```
precision.plot(kind = 'bar', title='emotions', figsize= (9,9))
plt.title('Precision of Emotions')
plt.xlabel('Emotions')
plt.ylabel('percent level')
```

```
In [ ]:
```

```
# Plotting recall of each emotions in each models
```

```
recall.plot(kind = 'bar', title='emotions', figsize= (9,9))
plt.title('Recall of Emotions')
plt.xlabel('Emotions')
plt.ylabel('percent level')
```

```
In [ ]:
```

```
# F1 score plot
```

```
score.plot(kind = 'bar', title='emotions', figsize= (9,9))
plt.title('F1 Score of Emotions')
plt.xlabel('Emotions')
```

```
plt.ylabel('percent level')
```